

Cloud Computing

Cloud Computing

Cloud computing refers to the delivery of computing services—including servers, storage, databases, networking, software, and more—over the internet. Instead of owning and maintaining physical servers or infrastructure, users can access these services on-demand from cloud service providers.



Traditional Vs Cloud Data Center

Consideration	On premises (Traditional)	Cloud
Upfront costs	Requires significant upfront investment.	Uses subscription, pay-as-you-go model.
Resource deployment	Resources managed and deployed in-house.	Deployment and management done on third-party infrastructure.
Security	Offers complete control of data and systems.	Offers high security but often doesn't meet specific regulatory requirements.
Compliance	Offers better visibility and storage options to meet compliance policies.	Doesn't offer complete authority and transparency into data storage and usage, which often violates compliance policies.
Scalability	Offers less flexibility due to hardware and space limitations.	Automates scalability as needed, often without intervention.
Availability	Vulnerable to single points of failure and can have more downtime.	Better prevention and downtime recovery with continuous monitoring and built-in redundant systems.

Cloud Essentials Characteristics

- On-demand Self Service
- Broad Network Access
- Resource Pooling and Multi-tenancy
- Scalability
- Measured Service



Cloud Essentials Characteristics

On-demand Self-Service: Cloud consumers can provision and manage computing resources, such as server instances, storage, and networking, without requiring human intervention from the service provider side. Users can provision resources as needed, typically using a web-based interface or API.

Broad Network Access: Cloud services are accessible over the network and can be accessed by any devices, such as laptops, smartphones, and tablets, using standard mechanisms. This accessibility promotes unlimited usage and allows access from anywhere with an internet connection.

Cloud Essentials Characteristics

Resource Pooling and Multi-tenancy: Cloud resources are shared among multiple users (multi-tenancy) while remaining dynamically assigned and reassigned based on demand. Physical and virtual resources are pooled to serve multiple consumers, allowing for cost efficiency and optimal resource utilization.

Scalability: Cloud services provide the capability to scale resources both vertically (increasing the power of individual resources) and horizontally (adding more instances of resources) to meet changing demand dynamically. This scalability enables users to rapidly scale up or down based on workload fluctuations.

Cloud Essentials Characteristics

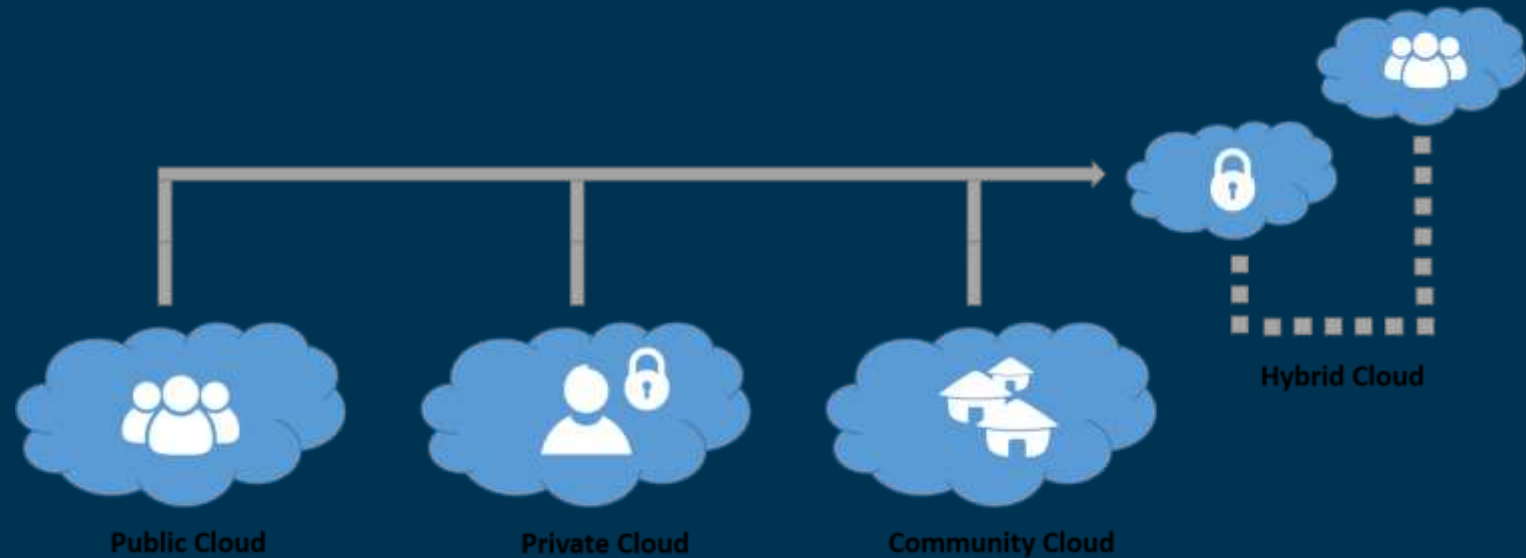
Measured Service: Cloud systems automatically control and optimize resource use by leveraging metering capabilities to monitor, measure, and report resource usage. This measurement facilitates transparent and optimized resource utilization while providing insights for billing, capacity planning, and performance optimization.

Types of Cloud Computing

Types of Cloud Computing

There are the following 4 types of cloud that you can deploy according to the organization's needs-

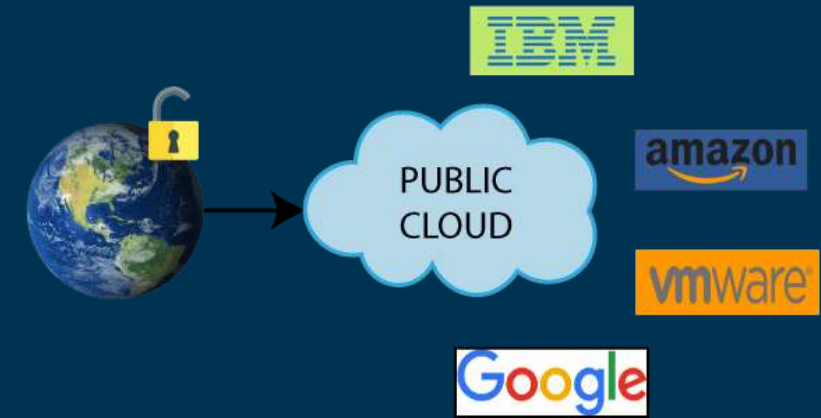
- Public Cloud
- Private Cloud
- Hybrid Cloud
- Community Cloud



Public Cloud

Public cloud is open to all to store and access information via the Internet using the pay-per-usage method.

- Public cloud is maintained by the cloud service provider.
- Public cloud is easier to integrate. Hence it offers a better flexibility approach to consumers.
- Public cloud is location independent because its services are delivered through the internet.
- Public cloud is highly scalable as per the requirement of computing resources.
- It is accessible by the general public, so there is no limit to the number of users.



Private Cloud

Private cloud is also known as an internal cloud or corporate cloud. It is used by organizations to build and manage their own data centers internally or by the third party. It can be deployed using Opensource tools such as Openstack and Eucalyptus.

- ❖ Private cloud provides a high level of security and privacy to the users.
- ❖ Private cloud offers better performance with improved speed and space capacity.
- ❖ It allows the IT team to quickly allocate and deliver on-demand IT resources.
- ❖ The organization has full control over the cloud because it is managed by the organization itself. So, there is no need for the organization to depends on anybody.
- ❖ It is suitable for organizations that require a separate cloud for their personal use and data security is the first priority

Hybrid cloud

- ❖ Hybrid cloud is a combination of public and private clouds.
- ❖ The main aim to combine these cloud (Public and Private) is to create a unified, automated, and well-managed computing environment.
- ❖ In the Hybrid cloud, non-critical activities are performed by the public cloud and critical activities are performed by the private cloud.
- ❖ Mainly, a hybrid cloud is used in finance, healthcare, and Universities.
- ❖ The best hybrid cloud provider companies are Amazon, Microsoft, Google, Cisco, and NetApp..

Community Cloud

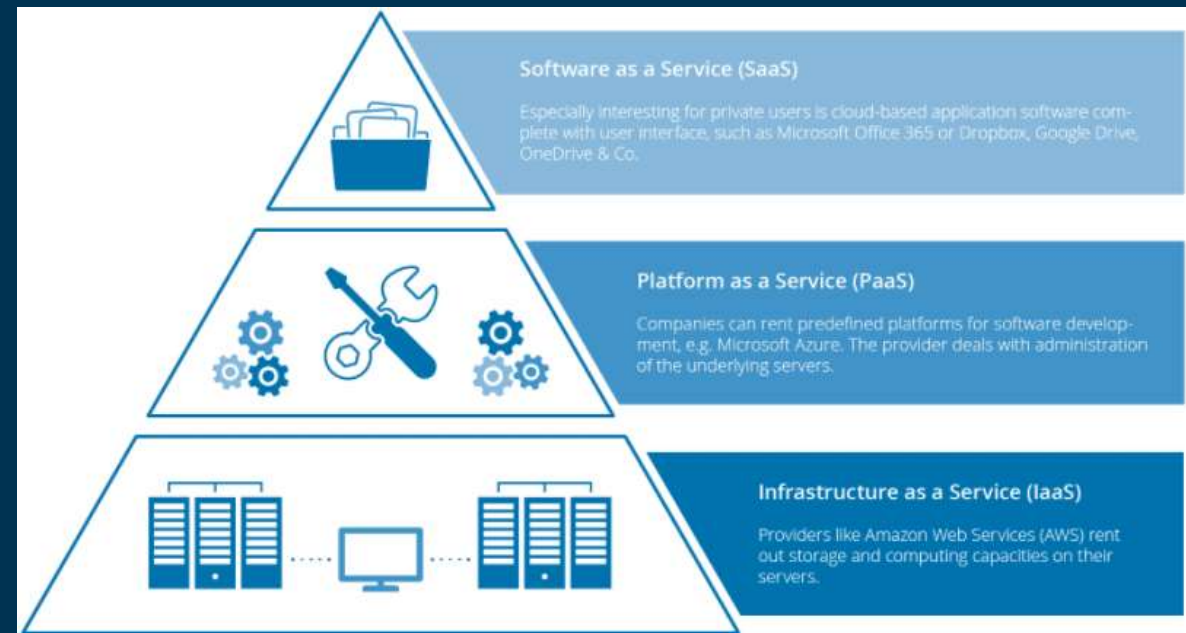
Community cloud allows systems and services to be accessible by a group of several organizations to share the information between the organization and a specific community. It is owned, managed, and operated by one or more organizations in the community, a third party, or a combination of them.

Cloud Service Models

Cloud Service Models

There are three Cloud Computing Service Models:

- Infrastructure as a Service (IaaS)
- Platform as a Service (PaaS)
- Software as a Service (SaaS)



Infrastructure as a Service (IaaS)

Cloud infrastructure services, known as Infrastructure as a Service (IaaS), are made of highly scalable and automated compute resources. IaaS is fully self-service for accessing and monitoring computers, networking, storage, and other services. IaaS allows businesses to purchase resources on-demand and as-needed instead of having to buy hardware outright.

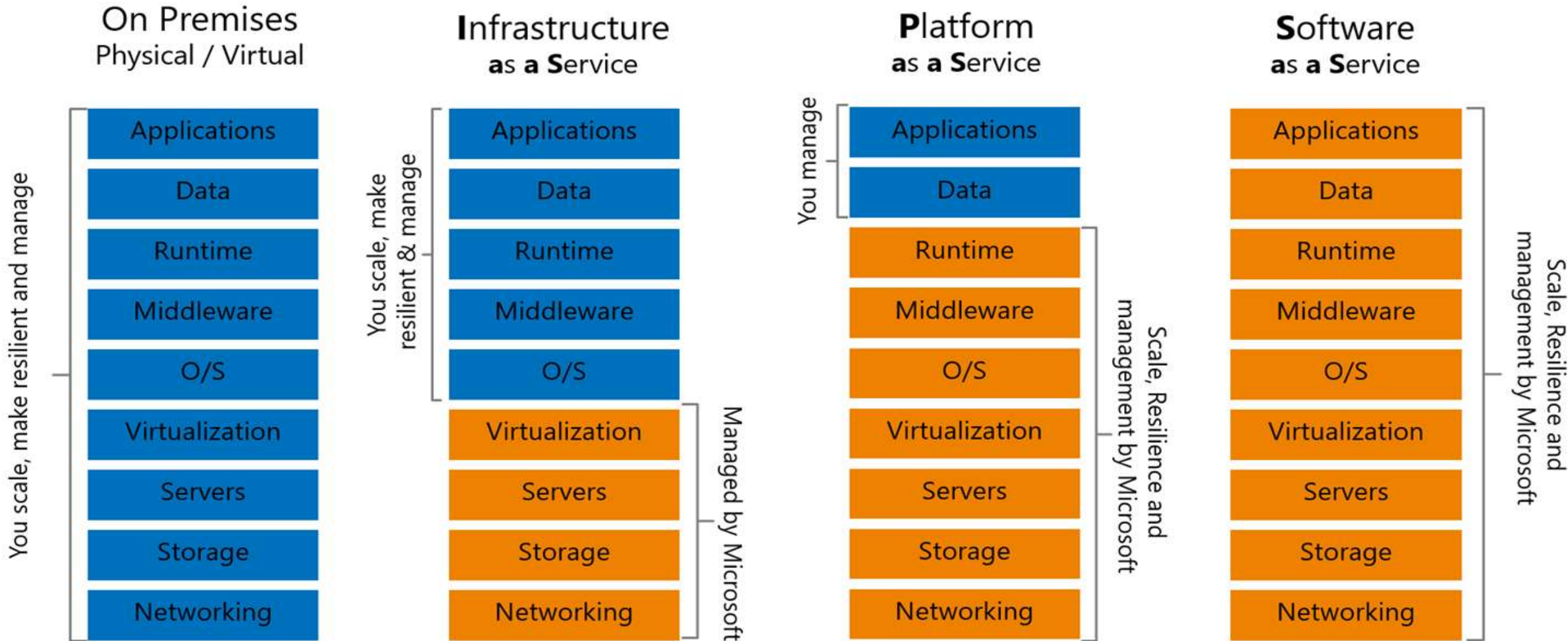
Platform as a Service (PaaS)

Platform As A Service (PAAS) is a cloud delivery model for applications composed of services managed by a third party. It provides elastic scaling of your application which allows developers to build applications and services over the internet and the deployment models include public, private and hybrid.

Software as a Service (SaaS)

Software As A Service (SAAS) allows users to run existing online applications and it is a model software that is deployed as a hosting service and is accessed over the internet or software delivery model during which software and its associated data are hosted centrally and accessed using their client, usually an online browser over the web. SAAS services are used for the development and deployment of modern applications.

Responsibility- Who owns What?





Amazon Web Services



Amazon Web Services

- AWS (Amazon Web Services) is a group of web services (also known as cloud services) being provided by Amazon since 2006.
- AWS provides huge list of services starting from basic IT infrastructure like CPU, Storage as a service, to advance services like Database as a service, Serverless applications, IOT, Machine Learning services etc..
- Hundreds of instances can be build and use in few minutes as and when required, which saves ample amount of hardware cost for any organizations and make them efficient to focus on their core business areas.
- Currently AWS is present and providing cloud services in more than 190 countries.
- Well-known for IaaS, but now growing fast in PaaS and SaaS.

Why AWS?

- **Low Cost:** AWS offers, pay as you go pricing. AWS models are usually cheapest among other service providers in the market.
- **Instant Elasticity:** You need 1 server or 1000's of servers, AWS has a massive infrastructure at backend to serve almost any kind of infrastructure demands, with pay for what you use policy.
- **Scalability:** Facing some resource issues, no problem within seconds you can scale up the resources and improve your application performance. This cannot be compared with traditional IT datacenters.
- **Multiple OS's:** Choice and use any supported Operating systems.
- **Multiple Storage Options:** Choice of high I/O storage, low cost storage. All is available in AWS, use and pay what you want to use with almost any scalability.

AWS Region

- An AWS Region is a cluster of data centers in a specific geographic area, such as the Northeastern United States or Western Europe. It is a best practice to choose a region that is geographically close to users; this reduces latency because data reaches the users more quickly.
- Each AWS Region includes multiple AZs. However, each AZ is restricted to a specific AWS Region. You can use multiple AZs within one Region, but you can't use the same AZ across multiple Regions.

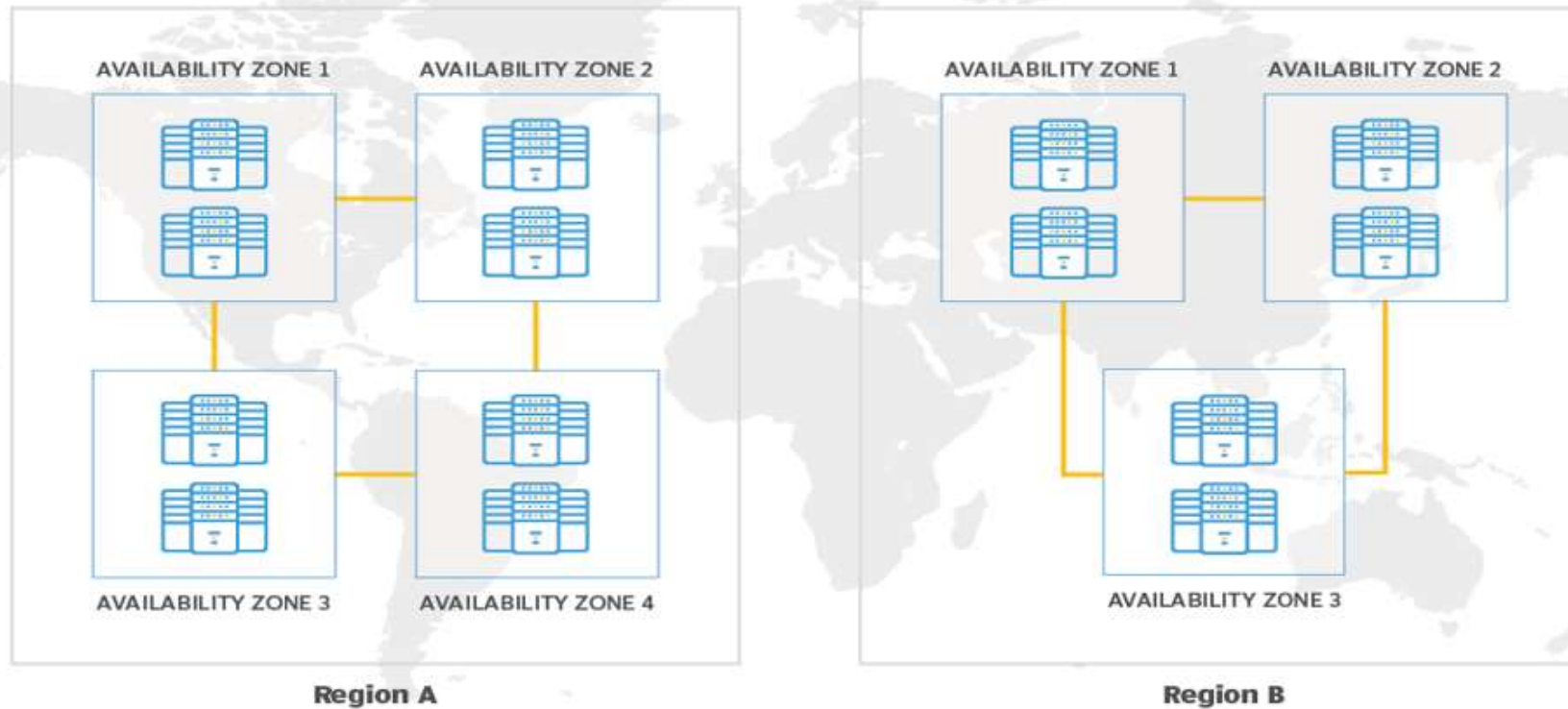
AWS Availability Zone

- An AZ is a standalone data center or set of data centers within a Region. Each AZ operates independently, so a failure in one won't affect others. In disaster recovery plans, enterprises use multiple AZs to increase redundancy and reliability.

AWS Global Infrastructure



Availability zones vs. regions



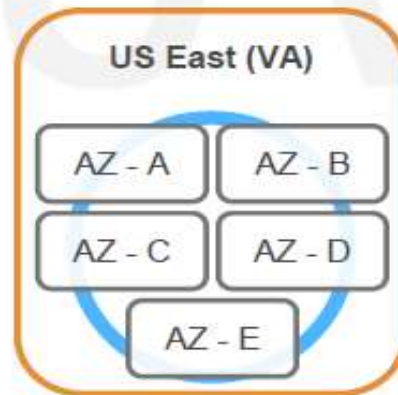
AWS Global Infrastructure

At least 2 AZs per region.

Examples:

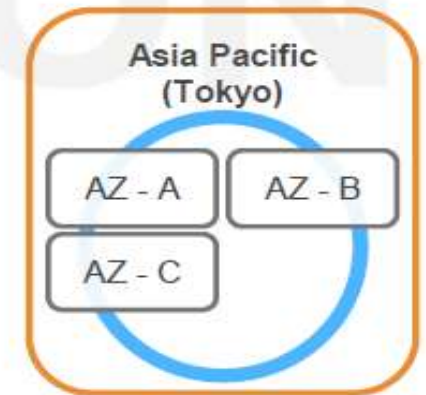
➤ US East (N. Virginia)

- us-east-1a
- us-east-1b
- us-east-1c
- us-east-1d
- us-east-1e



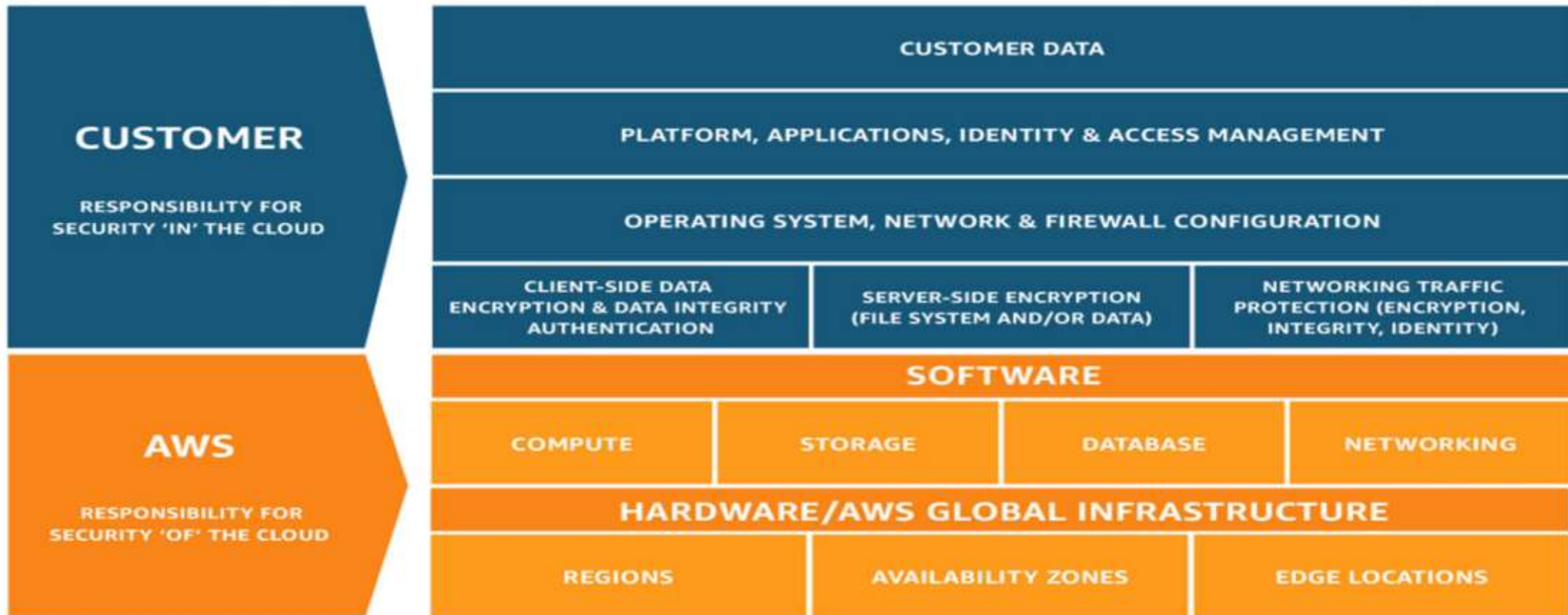
➤ Asia Pacific (Tokyo)

- ap-northeast-1a
- ap-northeast-1b
- ap-northeast-1c



Note: Conceptual drawing only. The number of Availability Zones (AZ) may vary.

AWS Responsibility





EC2

AWS Compute Services

Virtual Machines



EC2

What are the bare minimum infrastructure requirements for any application?

- Hardware (RAM, Processor, Processor type, HBA's, etc.)
- OS Disk & Data Disk
- OS Images
- Network
- Security (Firewall, Networking, Access Mechanism)
- Credentials

AWS Elastic Compute Cloud



EC2

- Amazon **EC2** stands for **Elastic Compute Cloud**, and is the Primary AWS web service.
- Provides Resizable compute capacity
- **Reduces the time required** to obtain and boot new server instances to minutes
- There are two key concepts to Launch instances in AWS:
 - **Instance Type**
 - **AMI**

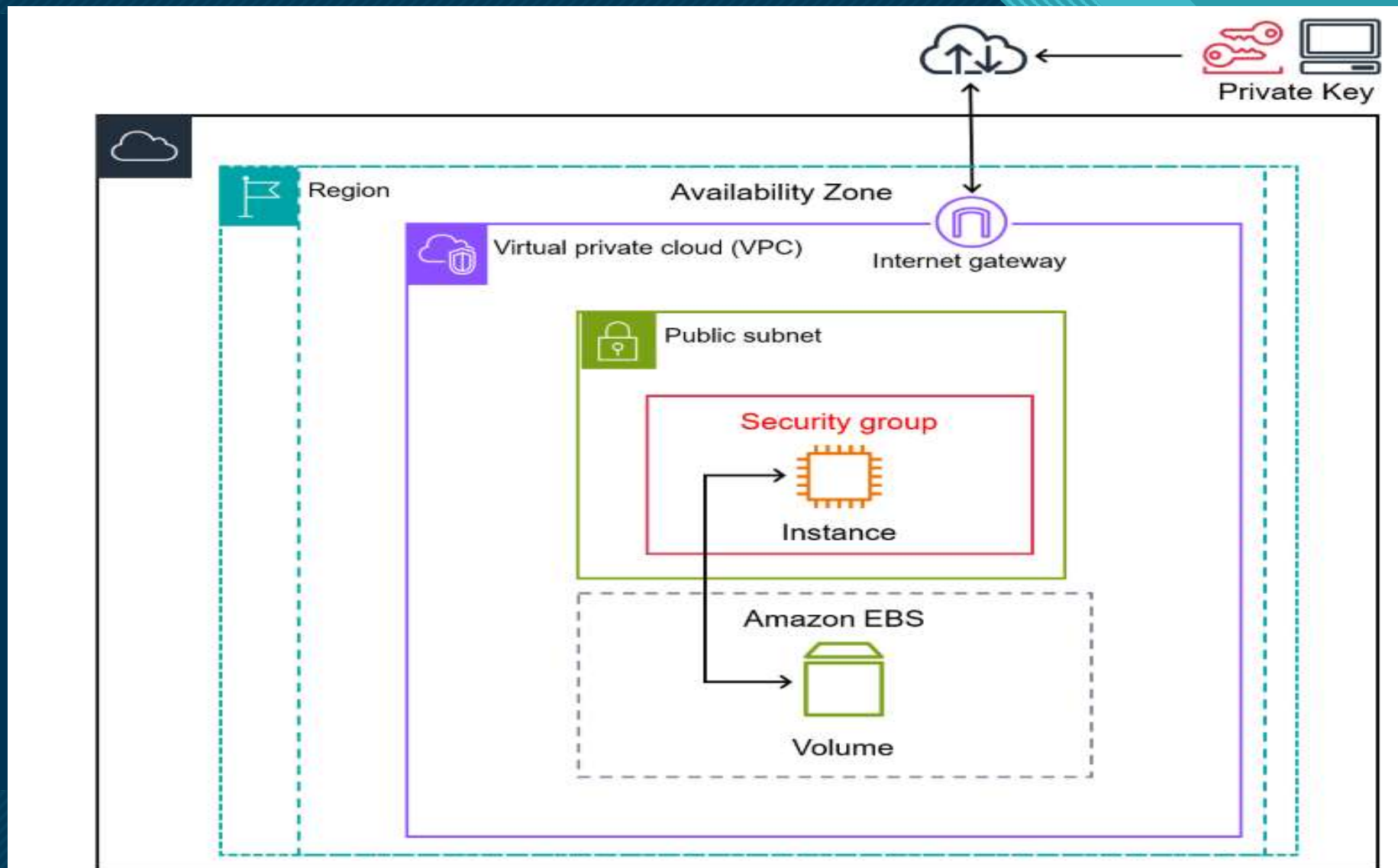
EC2 Facts:

- Scale capacity as your computing requirements change
- Pay only for capacity that you actually use
- Choose Linux or Windows OS as per need.
- **Deploy across AWS Regions and Availability Zones for reliability/HA**
- Only X86 based OS supported. So platform specific OS are not supported.

AWS Elastic Compute Cloud



EC2





EC2

Instance Types (EC2)

The instance type defines the different virtual hardware Models (sizes) supporting an Amazon EC2 instance.

- There are dozens of instance types available, varying in the following dimensions:
 - Virtual CPUs (vCPUs)
 - Memory
 - Storage (size and type)
 - Network performance
 - Graphical Processing
- Instance types are grouped into families based on the ratio of these values to each other.

Instance Types (EC2)



EC2

- EC2 instance types are optimized for different use cases and come in multiple sizes. This allows you to optimally scale resources to your workload requirements.
- AWS uses Intel® Xeon® processors for EC2 instances, providing customers with high performance and value.
- Consider the following when choosing your instances: Core count, memory size, storage size and type, network performance, and CPU technologies.
- Hurry Up and Go Idle - A larger compute instance can save you time and money, therefore paying more per hour for a shorter amount of time can be less expensive.

Instance Types with AWS



EC2

General purpose



Compute optimized



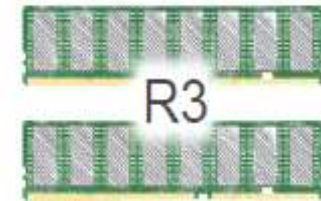
Storage and IO optimized



GPU enabled



Memory optimized



AMI's



EC2

AMI Stands for **Amazon Machine Image**

- **AMI is a template for the root volume** for the instance (example: an OS image, a webserver, an application server etc.)
- It also includes the launch permissions that control which AWS accounts can use the AMI to launch instances.
- All AMIs are based on x86 OSs, either Linux or Windows.

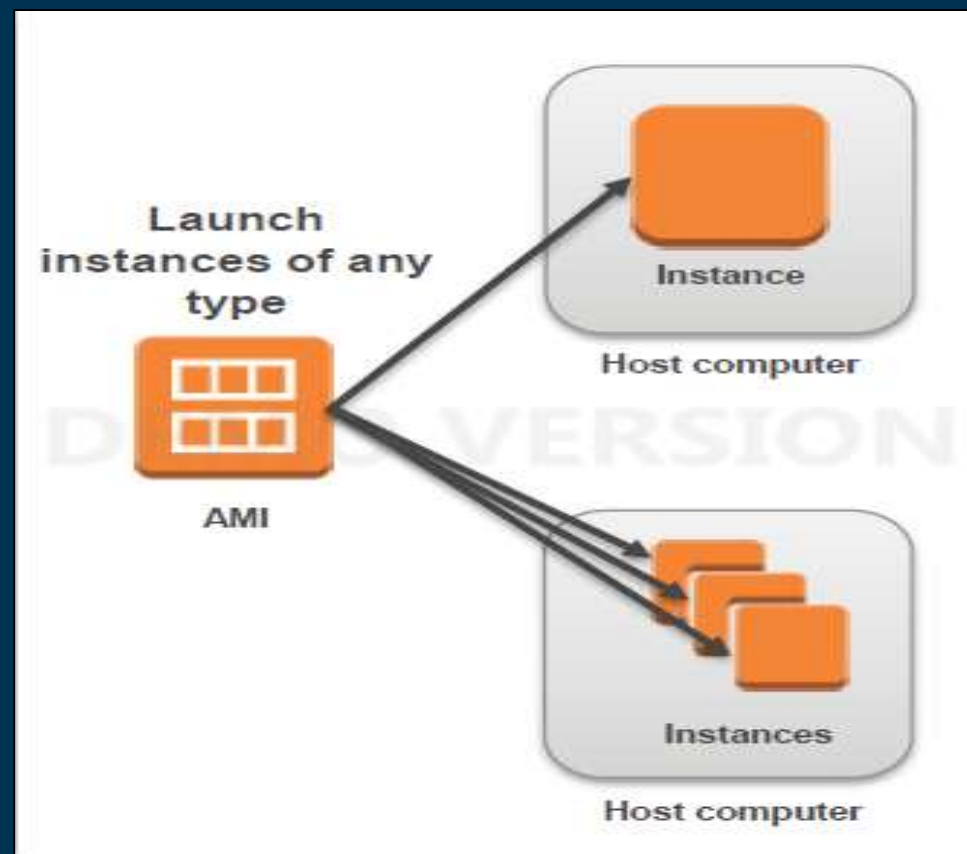


EC2

Instances and AMI's

Select an AMI based on:

- Region
- Operating system
- Architecture (32-bit or 64-bit)
- Launch permissions
- Storage for the root device
- Virtualization Type





EC2

Credentials

- Used to access an Instance.
- Amazon EC2 uses public-key cryptography to encrypt and decrypt login information.
- Public-key cryptography uses a public key to encrypt a piece of data and an associated private key to decrypt the data.
- These two keys together are called a *key pair*.
- AWS stores the public key, and the private key is kept by the customer. The private key is essential to acquiring secure access to an instance for the first time.
- When launching a Windows instance, Amazon EC2 generates a random password for the local administrator account and encrypts the password using the public key.

Launching EC2 Instance



EC2

- Procedure to launch EC2 instance in minutes:
- Determine the AWS Region in which you want to launch the Amazon EC2 instance.
- Launch an Amazon EC2 instance from a pre-configured Amazon Machine Image (AMI).
- Choose an instance type based on CPU, memory, storage, and network requirements.
- Configure network, security groups, storage volume, tags, and key pair; or directly launch instance post selecting AMI and Instance type.

LAB : Create an EC2 Linux Instance



EC2

Console Access: <https://console.aws.amazon.com/>

- Observe different available AMIs and Instance types
- Exercise:

Create a Linux Operating System using mandatory configurations:

- Instance Type : **T2.Micro**
 - AMI : **Amazon Linux 2 AMI**
 - Credentials: **Create/download your own key pair and use same.**
- Observe the instance details and navigate through the diff tabs
 - Reboot, Stop, Start your server
 - Access your instance using above downloaded key pair using different platforms i.e , instance connect , using putty , cli.

Note: (For the first instance take the default parameters wherever inputs required)



Instance Lifecycle



Network part



EC2

- **Private IP:** Alike traditional VMs/hosts, each EC2 instance must at least belong to one network (VPC in AWS) and must have at least one Private IP from that network/subnet.
- **Public IP:** A launched instance may also have a public IP address assigned. This IP address is assigned from the addresses reserved by AWS and cannot be specified. This IP address is unique on the Internet, persists only while the instance is running, and cannot be transferred to another instance.
- **Private/Public Domain Name System (DNS):** When you launch an instance, AWS
 - creates one private and one Public DNS name that can be used to access the instance. This DNS name is generated automatically and cannot be specified by the customer. This DNS name persists only while the instance is running and cannot be transferred to another instance.
- **Elastic IP (EIP):** An elastic IP address is an address unique on the Internet that you reserve independently and associate with an Amazon EC2 instance.



EC2

EC2 Security - Security Group

Virtual Firewall Protection

- AWS allows you to control traffic **in** and **out** of your instances through virtual firewalls called *security groups*.
- Security groups allow you to control traffic based on *port*, *protocol*, and *source(inbound)/destination(outbound)*.
- Security groups are associated with instances when they are launched. **Every instance must have at least one security group.** Though they can have more.

Amazon Server Storage



EC2

Amazon EBS (Elastic Block Store)

- Data stored on an Amazon EBS volume can persist independently of the life of the instance.
- Persistent **block-level storage volumes** for use with Amazon EC2 instances.
- 99% used volume type in AWS.

Amazon EC2 Instance Store

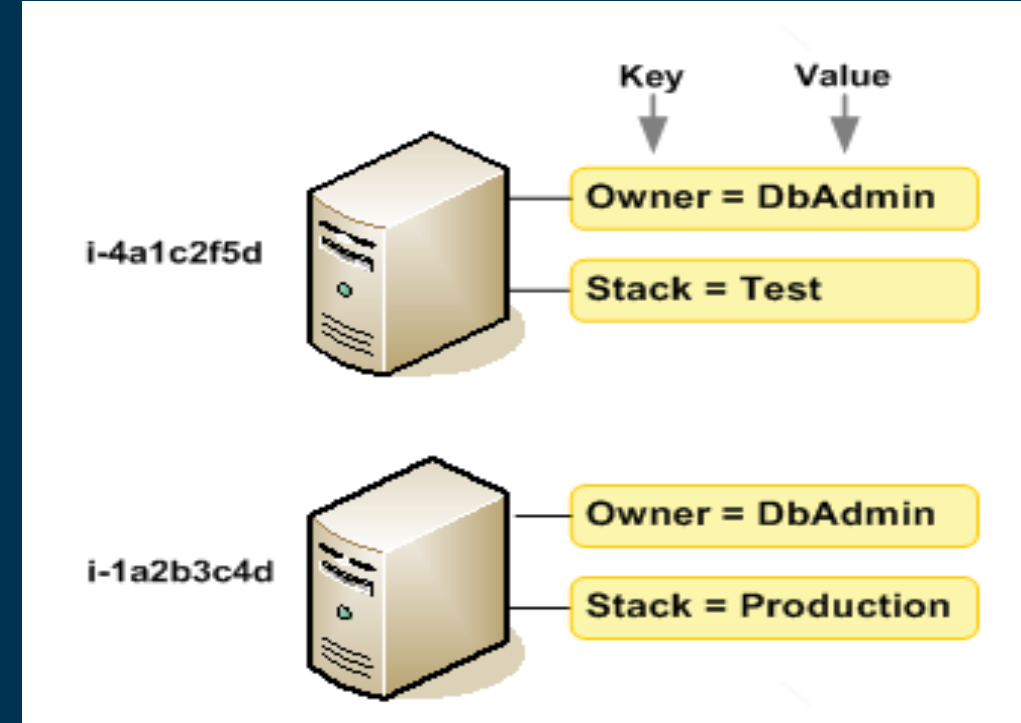
- Like Swap volumes attached to your server for faster processing.
- Data stored on a local instance store persists only as long as the instance is alive.
- Storage is ephemeral.
- Older volume type, used exceptionally in some instances i.e DB, High processing systems.

AWS Tags



EC2

- Tags help you manage your instances, images, and other Amazon EC2 resources.
- It's your own metadata to each resource in the form of *tags*.
- Each tag consists of a key and an optional value, both of which you define.
- It enables you to categorize your AWS resources in different ways, for example, by purpose, owner, or environment.
- You can organize your billing information based on resources that have the same tag key values.



AWS Tags



EC2

- Maximum number of tags per resource—50
- Tag keys and values are case-sensitive.
- You can't terminate, stop, or delete a resource based solely on its tags; you must specify the resource identifier : ARN's
- Not all resources supports tags. Pls check the supported list from AWS website.



amazon
S3

AWS S3

What is Amazon S3?

Amazon S3, or Simple Storage Service, is a scalable object storage service offered by Amazon Web Services (AWS). It is designed to store and retrieve any amount of data from anywhere on the web and is a fundamental building block of cloud computing.

Key Characteristics

- **Scalability:** Amazon S3 can scale virtually infinitely, allowing users to store and retrieve any amount of data.
- **Durability and Reliability:** It is designed to provide 99.999999999% (11 9's) durability of objects over a given year.
- **Object Storage:** S3 is object-based storage, where data is stored as objects. Each object consists of data, a key, and metadata.
- **Global Accessibility:** S3 allows users to access their data from anywhere on the web, making it suitable for a global audience.

Data Storage Options

- **Storage Classes:** Amazon S3 offers different storage classes, each designed for different use cases. These include Standard, Intelligent-Tiering, Glacier for archiving, and others.
- **Versioning:** S3 supports versioning, allowing users to keep multiple versions of an object in the same bucket.

Use Cases



- Data Backup and Restore: S3 provides a secure and reliable solution for data backup and restoration.
- Data Archiving: Its storage classes, such as Glacier, are suitable for long-term data archiving.
- Static Website Hosting: S3 can be used to host static websites with low-latency access to content.
- Data Lakes: S3 serves as a popular choice for building data lakes due to its scalability and durability.
- Big Data Analytics: It integrates seamlessly with various AWS analytics services for processing large datasets.

Benefits



- **Cost-Effective:** Users pay only for the storage they use, and S3's pricing is designed to be cost-effective for various workloads.
- **Scalability:** S3 can seamlessly scale to accommodate growing amounts of data.
- **Security:** It offers features like access control, encryption, and compliance with various standards.

Bucket



- A bucket is a container for storing objects. Objects in Amazon S3 are essentially files, and each file must be stored in a bucket. Think of a bucket as a top-level folder or directory within Amazon S3.



Characteristics of S3 buckets

- **Naming**
- **Region**
- **Access Control**
- **Storage Classes**
- **Versioning**
- **Lifecycle Policies**
- **Server Access Logging**

How to create Bucket

➤ **Step 1: Sign in to AWS Console**

Go to the AWS Management Console and sign in with your AWS account credentials.

➤ **Step 2: Open S3 Console**

Navigate to the Amazon S3 service in the AWS Management Console.

➤ **Step 3: Click "Create Bucket"**

Click on the "Create Bucket" button to initiate the bucket creation process.

➤ **Step 4: Configure Bucket Settings**

Enter a unique and meaningful name for your bucket.

Choose a region for your bucket to optimize latency, durability, and costs.

➤ **Step 5: Configure Additional Options**

Set up versioning if needed.

Configure server access logging for auditing.

➤ **Step 6: Set Permissions**

Define access controls for the bucket, specifying who can access it and how.

➤ **Step 7: Review and Create**

Review your configuration settings and click "Create Bucket."

Uploading and Downloading Objects



- Open the S3 console and navigate to the desired bucket.
- Click on "Upload" and select the files or folders you want to upload.
- Configure any additional settings, such as storage class or metadata.
- Click "Upload" to transfer the files to your bucket.

Setting Permissions

➤ **Bucket Policies**

- Define policies that control access to your bucket based on specified conditions.

➤ **Access Control Lists (ACLs)**

- Set granular access permissions for individual objects within your bucket.

➤ **Identity and Access Management (IAM)**

- Use IAM roles and policies to control access to your S3 resources.

Identity and Access Management



AWS Identity and Access Management (IAM)

- AWS Identity and Access Management (IAM) is a web service provided by Amazon Web Services (AWS) that enables you to manage access to AWS services and resources securely. IAM allows you to control who (users) can do what (actions) on which resources (AWS resources) and provides fine-grained access control.



IAM Key Roles



1. User Management
2. Group Management
3. Role-based Access Control (RBAC)
4. Policy Management
5. Fine-Grained Access Control
6. Security Best Practices
7. Integration with AWS Services

User Management



- IAM enables the creation and management of AWS users within your AWS account. Each user can have individual security credentials, such as access keys, passwords, or Multi-Factor Authentication (MFA) devices.
- You can create IAM users to distribute your work among team/users.
- Different permissions can be given to IAM users

LAB : IAM User



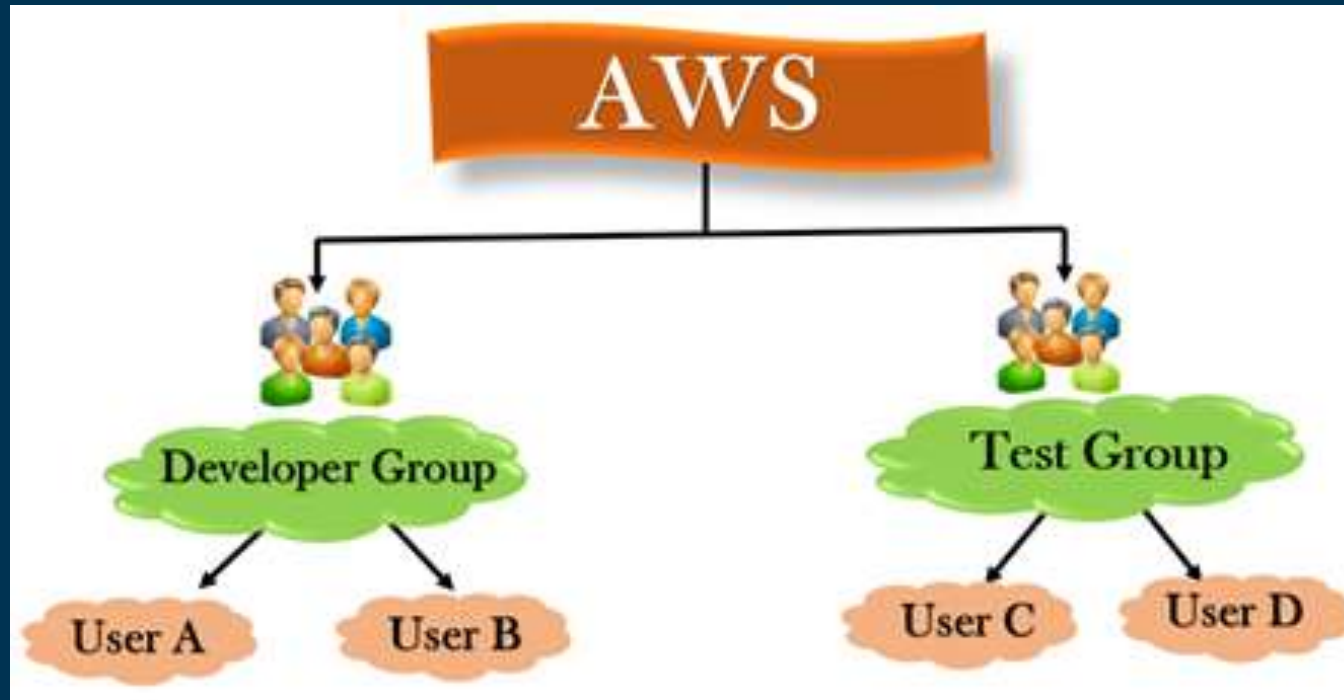
In this exercise, you will create an IAM user who can perform all administrative IAM functions. Then you will log in as that user so that you no longer need to use the root user login. Using the root user login only when explicitly required is a recommended security practice (along with adding MFA to your root user).

1. While logged in as the root user, create a new IAM user called user1.
2. Provide Access type -> Programmatic access and Management console access
3. Set a console password and click next
4. Logout from root and Use the customized sign-in link to sign in as Administrator
5. Try to login and check if you are able to access the s3 buckets in new user.
6. Were to able to see? What about other services?

Group Management



- Users can be organized into groups, simplifying the management of permissions. Policies are attached to groups, allowing multiple users to inherit the same permissions.



LAB : IAM Group



In this exercise, you will create a group for all IAM administrator users and assign the proper permissions to the new group. This will allow you to avoid assigning policies directly to a user later in these exercises.

1. Log in as the root user.
2. Go to **Security, Identity & Compliance** → **IAM** → **Groups**
3. Create an IAM group provide name Elevated_Users
4. Don't select any policy and click create.
5. Add user created to the group

Role-based Access Control (RBAC)



Role-based access control (RBAC) is a security methodology based on managing user access to protect resources, including data, applications, and systems, from improper access, modification, addition, or deletion. RBAC grants access based on a user's needs according to their position. Roles and associated privileges are defined, and permissions are assigned to control access (i.e., what a user can see), the scope of approved operations (i.e., what a user can do, such as view, create, or modify), and session time (i.e., how long a user can have access).

IAM Policy



An IAM policy in AWS Identity and Access Management (IAM) is a JSON (JavaScript Object Notation) document that defines permissions. It specifies the actions allowed or denied, resources to which the actions apply, and the conditions under which actions are allowed or denied.



Components of an IAM Policy

```
{
  "Version": "2012-10-17",
  "Id": "PutObjPolicy",
  "Statement": [{
    "Sid": "DenyObjectsThatAreNotSSEKMS",
    "Principal": "*",
    "Effect": "Deny",
    "Action": "s3:PutObject",
    "Resource": "arn:aws:s3:::sandeepbucket.com/*",
    "Condition": {
      "Null": {
        "s3:x-amz-server-side-encryption-aws-kms-key-id": "true"
      }
    }
  }]
}
```

- **Effect:** Specifies whether the policy allows or denies the defined permissions. It can have two values: "Allow" or "Deny".
- **Action:** Describes the specific API actions that the policy allows or denies. For example, s3:GetObject, ec2:StartInstances, etc.
- **Resource:** Indicates the AWS resources to which the actions apply. It specifies the ARN (Amazon Resource Name) of the resource. For instance, an S3 bucket, an EC2 instance, an IAM user, etc.
- **Condition (Optional):** Defines the circumstances under which the policy is in effect. Conditions can include factors such as the source IP address, time of day, whether Multi-Factor Authentication (MFA) is used, and more

LAB : IAM Policies



Create an IAM Policy

In this exercise, you will create an IAM Policy, which will provide EC2 list and Read permissions to the policy holder

1. While logged in as the root user, Click on IAM → Policies.
2. Select S3Fullaccess policy or Click Create a policy
3. Select Service, Action, Resources.
4. Review Policy and click Create policy
5. View your policy in search bar.
6. Click on Action → Attach, and select the New user
7. Go to Users tab and see the new policy attached to user
8. Now try to view the s3 bucket features . Are you able to ?
9. Remove the policy .
10. Create a policy and then try to attach it to user.

AWS Identity and Access Management (IAM)



- Delete AWS account (root) access keys.
- Create individual IAM users.
- Use groups to assign permissions to IAM users.
- Grant least privilege.
- Configure a strong password policy.
- Enable MFA for privileged users.



Amazon RDS: Managed Relational Databases

What Is Data ?



Data is a collection of a distinct unit of information. This “data” is used in a variety of forms of text, numbers, media and many more. Talking in terms of computing. Data is basically information that can be translated into a particular form for efficient movement and processing.

What Is Database ?



The database is an organized collection of structured data to make it easily accessible, manageable and update. In simple words, you can say, a database is a place where the data is stored. The best analogy is the library. The library contains a huge collection of books of different genres, here the library is database and books are the data.



Why Are Databases Important?



Databases are essential in various fields and industries due to their fundamental role in organizing, storing, retrieving, and managing data. Here are several reasons why databases are important:

1. **Data Organization**
2. **Data Integrity and Consistency.**
3. **Data Retrieval and Manipulation**
4. **Improved Decision Making**
5. **Data Security and Access Control**
6. **Scalability and Performance**
7. **Support for Transactions**
8. **Support for Concurrent Access**
9. **Backup and Recovery**
10. **Regulatory Compliance**

Types of Databases



Databases can be categorized into various types based on their data model, structure, and usage. Some of the common types of databases include:

1.Relational Databases (RDBMS):

1. Structured Query Language (SQL) is used to manage data in tables with predefined schemas.
2. Examples: MySQL, PostgreSQL, Oracle, SQL Server, SQLite.

2.NoSQL Databases:

1. These databases handle unstructured or semi-structured data and offer more flexible schemas.
2. Types include document-oriented, key-value stores, column-oriented, and graph databases.
3. Examples: MongoDB (document-based), Redis (key-value), Cassandra (column-oriented), Neo4j (graph).

3.Object-Oriented Databases:

1. Store data in the form of objects similar to those used in object-oriented programming.
2. They support complex data types and relationships directly.
3. Examples: ObjectDB, db4o.

Cloud vs Traditional Database



Cloud Database



Traditional Database





Cloud vs Traditional Database

Parameter	Cloud Database	Traditional Database
Deployment	Cloud databases are hosted and managed by cloud service providers (CSPs) on their infrastructure, accessible over the internet.	Traditional databases are installed and maintained on-premises or within an organization's local infrastructure.
Architecture	Utilizes distributed computing and virtualization technologies, enabling scalability and fault tolerance.	Often based on a client-server model where the database server manages and stores the data, and clients access it via a network.
Hardware & Scalability	Cloud databases are inherently scalable, allowing easy horizontal or vertical scaling based on demand without significant downtime.	Scaling usually requires hardware upgrades or additional servers, which may involve downtime and higher costs.
Management & Maintenance	CSPs handle most maintenance tasks like updates, backups, security, and infrastructure management, reducing the burden on users.	Organizations are responsible for managing database servers, updates, backups, security, and ensuring high availability.
Cost Structure	Follows a pay-as-you-go or subscription-based model, where users pay for usage, storage, and additional features. No upfront hardware costs.	Initial setup costs for hardware, software licenses, and ongoing maintenance expenses are directly borne by the organization.
Control	Full control over Database	Offers full control and customization over the infrastructure and database configuration.
Accessibility	Offers accessibility from anywhere with an internet connection, allowing easier collaboration and remote work.	Access is typically restricted to the local network, requiring VPN or specific configurations for remote access.

What is Amazon RDS?



- Amazon RDS is a **Relational Database Service** by Amazon Web Services.
- A relational database is a type of database that stores data in tables with rows and columns.
- Amazon RDS provides an industry-standard relational database that is cost-efficient with resizable capacity.
- It gives users solutions for easier setup, operation, and scale of a relational database in the cloud.
- Amazon RDS is a service that simplifies the setup, operations, and scaling of a relational database on AWS.
- With Amazon RDS, you can spend more time focusing on the application and the schema and let Amazon RDS offload
- common tasks like backups, patching, scaling, and replication.

Benefits of Amazon RDS



AWS RDS Database Engines



RDS is not a database, it's a service that manages databases and there are six database engines which RDS provides, and they are:



AWS Relational Databases



- With Amazon RDS, you can accelerate your development timelines and establish a consistent operating model for managing relational databases. For example, Amazon RDS makes it easy to replicate your data to increase availability, improve durability, or scale up or beyond a single database instance for read-heavy database workloads.
- With Amazon RDS, you can typically use the same tools to query, analyze, modify, and administer the database.
- The range of DB Instance classes extends from a db.t2.micro with 1 virtual CPU (vCPU) and 1 GB of memory, up to a db.r3.8xlarge with 32 vCPUs and 244 GB of memory. As your needs change over time, you can change the instance class and the balance of compute of memory, and Amazon RDS will migrate your data to a larger or smaller instance class.

AWS Relational Databases

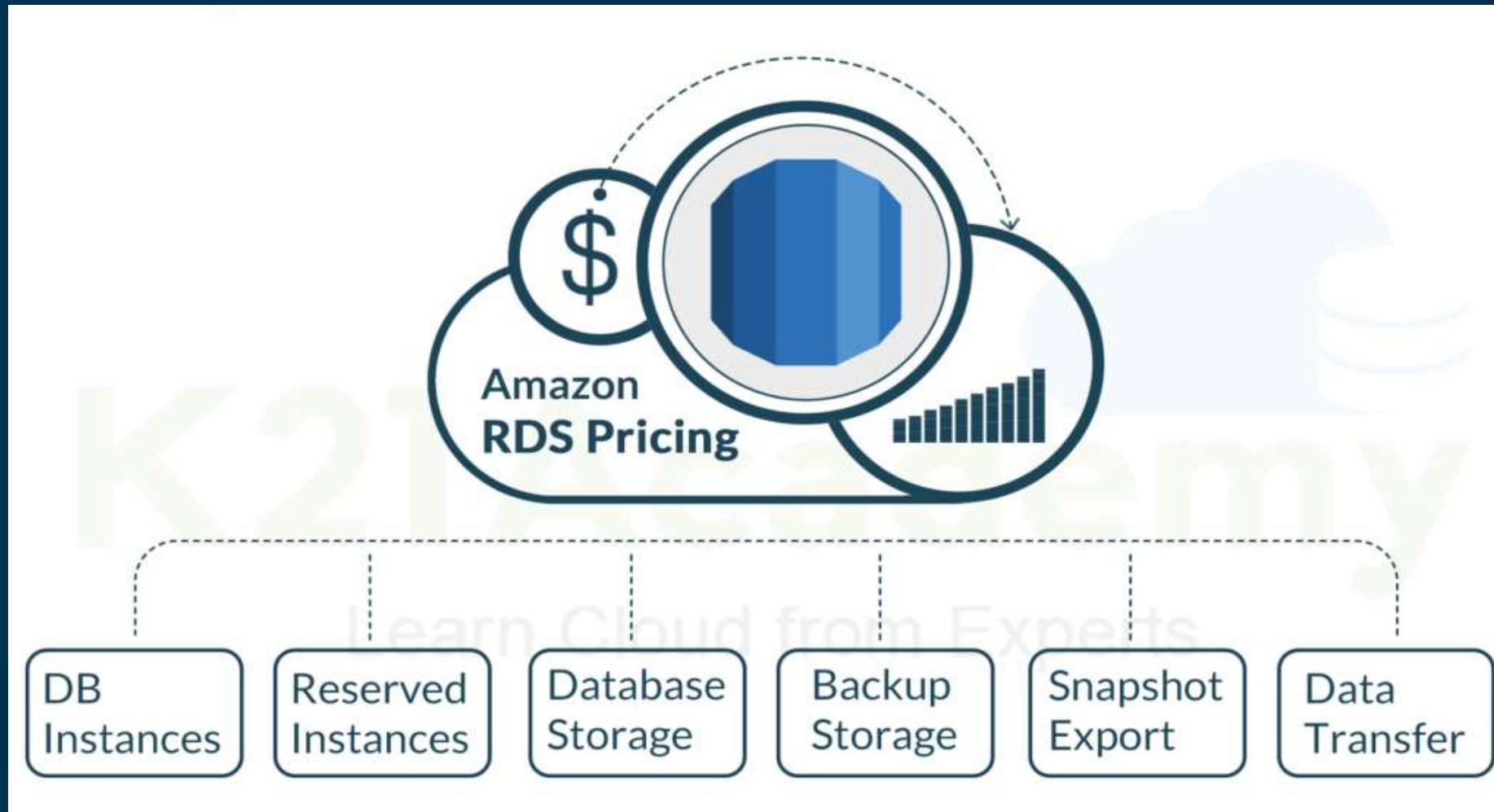


- You can also choose to run nearly any database engine using Windows or Linux Amazon Elastic Compute Cloud (Amazon EC2) instances and manage the installation and administration yourself.
- As RDS is a managed Service (Paas), Amazon doesn't provide shell access for RDS DB instances.
- You can have automated backups performed when you need them, or manually create your own backup snapshot.

Licensing:

- AWS offers two licensing models: **License Included and Bring Your Own License (BYOL).**
- License Included In the License Included model, the license is held by AWS and is included in the Amazon RDS instance price.
- Bring Your Own License (BYOL) In the BYOL model, you provide your own license

Amazon RDS Pricing



Creating RDS Instance



Create a SQL Amazon RDS Instance

1. Log in to the AWS Management Console, and navigate to the Amazon RDS Console.
2. Launch a new Amazon RDS DB Instance, and select MySQL Community Edition instance as the database engine.
3. Configure the DB Instance to use Multi-AZ and General Purpose (SSD) storage.

Warning: This is not eligible for AWS Free Tier; you will incur a small charge by provisioning this instance.

4. Set the DB Instance identifier and database name to **MySQL123**, and configure the master username and password.
5. Validate the configuration settings, and launch the DB Instance.
6. Return to the list of the Amazon RDS instances. You will see the status of your Amazon RDS database as Creating. It may take up to 20 minutes to create your new Amazon RDS instance.

RDS AZ Failover testing



Simulate a Failover from One AZ to Another

In this exercise, you will use Multi-AZ failover to simulate a failover from one Availability Zone to another.

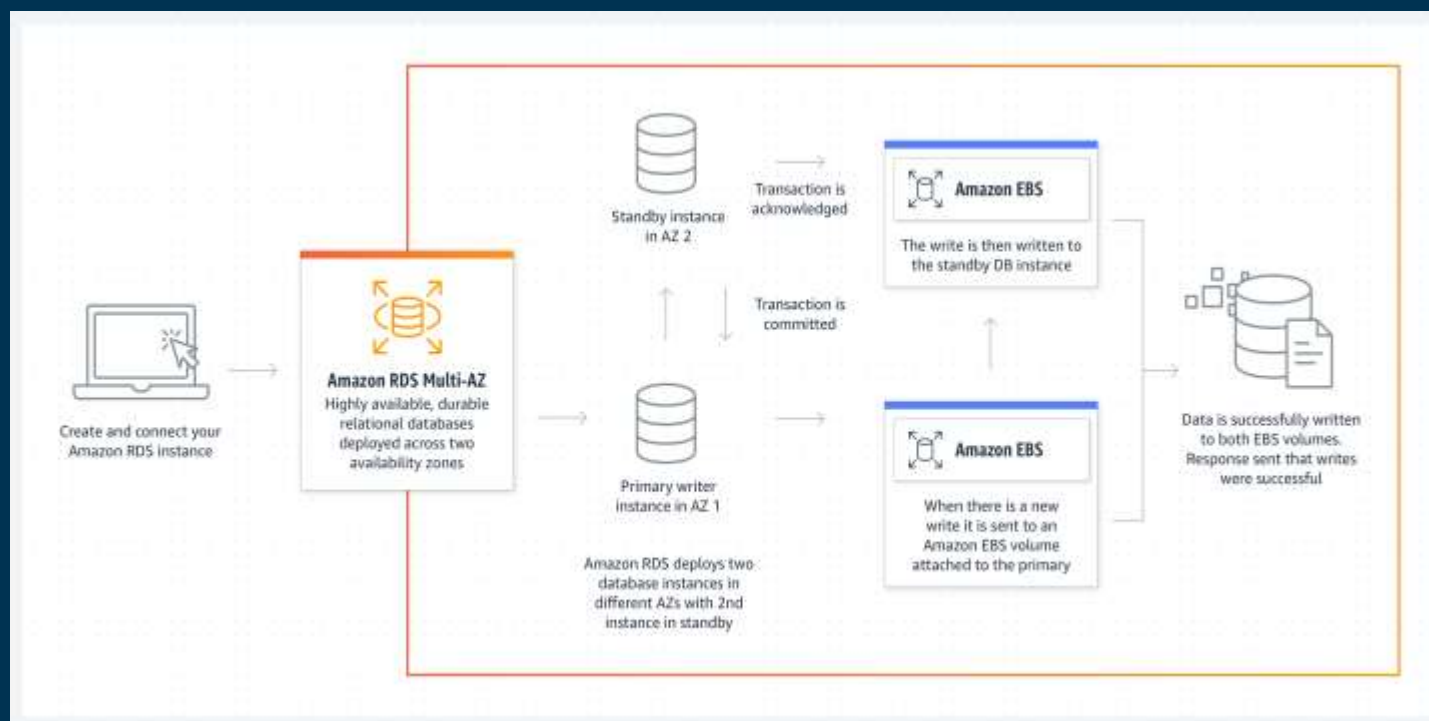
1. In the Amazon RDS Console, view the list of DB Instances.
2. Find your DB Instance called MySQL123, and check its status. When its status is Available, proceed to the next step.
3. Select the instance, and issue a Reboot command from the actions menu.
4. Confirm the reboot.

You have now simulated a failover from one Availability Zone to another using Multi-AZ failover. The failover should take approximately two or three minutes.

Multi-AZ deployment



In an Amazon RDS Multi-AZ deployment, Amazon RDS automatically creates a primary database (DB) instance and synchronously replicates the data to an instance in a different AZ. When it detects a failure, Amazon RDS automatically fails over to a standby instance without manual intervention.



Read Replica



- Replication means a continuous copy of the data from one Database (master/publisher) to another Database (slave/subscriber).
- The slave/subscriber Databases can be on the same server or different servers and contain the replica of the master database.
- The main aim of replication is to provide fault-tolerant access to data in case of failure of the master. Also, load balancing and routing some of the queries to the subscriber database to reduce the load on the primary Database.
- Amazon RDS Read Replication is a feature on Amazon RDS that allows you to create one or more read copies of the primary Database within the same or different AWS region. The data from the master database is asynchronously copied to these slave or secondary databases.



Multi-AZ deployment vs Read Replica

Multi-AZ Deployment

High Availability: Provides failover support in case of primary database failure by maintaining a synchronous standby replica in a different Availability Zone (AZ).

Automatic creation of a synchronous standby replica in a different AZ from the primary database.

Synchronous replication between primary and standby replicas, ensuring data consistency at all times.

Automatic failover to the standby replica in case of primary database failure, with minimal downtime.

Ensures high availability and data durability by maintaining synchronized copies in different AZs.

Generally more expensive due to the cost of maintaining standby replicas in a different AZ.

Minimal performance impact on the primary instance due to synchronous replication.

Critical production databases requiring high availability and minimal downtime in case of failure.

Read Replicas

Scaling: Used for offloading read-only queries from the primary database to improve performance and scalability.

Manual creation of one or more asynchronous replicas from the primary database.

Asynchronous replication from the primary database, providing slightly delayed data compared to the primary.

No automatic failover capability. If the primary instance fails, manual intervention or automated scripts are needed to promote a replica to become the new primary.

Used primarily for scaling read-heavy workloads, reporting, analytics, etc., without impacting the primary database's performance.

Additional cost-effective option for scaling read operations without the expense of maintaining synchronous replicas.

Read replicas can reduce the load on the primary instance, improving overall performance for read-heavy workloads.

Applications with read-heavy workloads that can benefit from distributing read queries to replicas.

AWS PostgreSQL



- PostgreSQL is an advanced, enterprise-class, and open-source relational database system. PostgreSQL supports both SQL (relational) and JSON (non-relational) querying.
- PostgreSQL is a highly stable database backed by more than 20 years of development by the open-source community.
- PostgreSQL is used as a primary database for many web applications as well as mobile and analytics applications.
- PostgreSQL's community pronounces PostgreSQL
- AWS PostgreSQL refers to PostgreSQL database hosting services provided by Amazon Web Services (AWS) through their Amazon Relational Database Service (RDS). PostgreSQL is an open-source, powerful, and feature-rich relational database management system known for its reliability, extensibility, and strong compliance with SQL standards.

Lab : Setup AWS PostgreSQL



We will complete this lab in three steps:

Step 1: we will create one AWS EC2

Step 2 : We will setup PostgreSQL

Step 3: Connect EC2 Instance to PostgreSQL RDS

Step 1: Create one AWS EC2



1. Log in to your AWS Management Console.
2. Navigate to the EC2 Dashboard.
3. Click on "Launch Instance" to create a new EC2 instance.
4. Choose an Amazon Machine Image (AMI) for your instance.
5. Select an instance type and configure the instance details.
6. Configure security groups to allow inbound traffic on the PostgreSQL port (default is 5432).
7. Launch the instance and create or select an existing key pair to access it securely.

Step 2: Setup PostgreSQL



1. Go to the Amazon RDS service in the AWS Management Console.
2. Click on "Create database" and choose PostgreSQL as the database engine.
3. Select the desired version, instance size, and other configuration settings.
4. Set up the database credentials (username and password) and define the database details.
5. Configure the security group to allow inbound traffic from the EC2 instance on port 5432 (PostgreSQL default port).
6. Launch the RDS instance

Step 3: Connect EC2 Instance to RDS



To connect to a PostgreSQL DB instance using psql

1. Find the endpoint (DNS name) and port number for your DB instance.
 - a. Sign in to the AWS Management Console and open the Amazon RDS console at <https://console.aws.amazon.com/rds/>.
 - b. In the upper-right corner of the Amazon RDS console, choose the AWS Region for the DB instance.
 - c. In the navigation pane, choose **Databases**.
 - d. Choose the PostgreSQL DB instance name to display its details.
 - e. On the **Connectivity & security** tab, copy the endpoint. Also note the port number. You need both the endpoint and the port number to connect to the DB instance.
2. Connect to the EC2 instance that you created earlier by following the steps in [Connect to your Linux instance](#) in the *Amazon EC2 User Guide for Linux Instances*.

```
ssh -i location_of_pem_file ec2-user@ec2-instance-public-dns-name
```

3. To install the psql command-line client from PostgreSQL on Amazon Linux 2023, run the following command:

```
sudo dnf install postgresql15
```

And then execute below command to connect your db to ec2

```
psql --host=endpoint --port=5432 --dbname=postgres --username=postgres
```

Introduction of NoSQL



1. NoSQL is a type of database management system (DBMS) that is designed to handle and store large volumes of unstructured and semi-structured data.
2. Unlike traditional relational databases that use tables with pre-defined schemas to store data, NoSQL databases use flexible data models that can adapt to changes in data structures and are capable of scaling horizontally to handle growing amounts of data.
3. The term NoSQL originally referred to “non-SQL” or “non-relational” databases, but the term has since evolved to mean “not only SQL,” as NoSQL databases have expanded to include a wide range of different database architectures and data models.

Introduction of NoSQL



NoSQL databases are generally classified into four main categories:

1. **Document databases:** These databases store data as semi-structured documents, such as JSON or XML, and can be queried using document-oriented query languages.
2. **Key-value stores:** These databases store data as key-value pairs, and are optimized for simple and fast read/write operations.
3. **Column-family stores:** These databases store data as column families, which are sets of columns that are treated as a single entity. They are optimized for fast and efficient querying of large amounts of data.
4. **Graph databases:** These databases store data as nodes and edges, and are designed to handle complex relationships between data.

SQL vs NoSQL

nosql

SQL

- ✓ Stands for Structured Query Language
- ✓ Relational database management system (RDBMS)
- ✓ Suitable for structured data with predefined schema
- ✓ Data is stored in tables with columns and rows
- ✓ Follows ACID properties (Atomicity, Consistency, Isolation, Durability) for transaction management
- ✓ Supports JOIN and complex queries
- ✓ Uses normalized data structure
- ✓ Requires vertical scaling to handle large volumes of data
- ✓ Examples: MySQL, PostgreSQL, Oracle, SQL Server, Microsoft SQL Server

NoSQL

- ✓ Stands for Not Only SQL
- ✓ Non-relational database management system
- ✓ Suitable for unstructured and semi-structured data
- ✓ Data is stored in collections or documents
- ✓ Does not necessarily follow ACID properties
- ✓ Does not support JOIN and complex queries
- ✓ Uses denormalized data structure
- ✓ Horizontal scaling is possible to handle large volumes of data
- ✓ Examples: MongoDB, Cassandra, Couchbase, Amazon DynamoDB, Redis



AWS Networking

AWS Virtual Private Cloud



- VPC stands for Virtual Private Cloud. VPC is a private cloud located inside a public cloud that enables you to experience the benefits of a virtualized network while using public cloud resources.
- Amazon Virtual Private Cloud (Amazon VPC) provides a logically isolated area of the AWS cloud where you can launch AWS resources in a virtual network that you define.
- You have complete control over your virtual networking environment, including a selection of your IP address range, the creation of subnets, and configuration of route tables and network gateways.
- You can easily customize the network configuration for your Amazon Virtual Private Cloud. For example, you can create a public-facing subnet for web servers that can access to the internet and can also place your backend system such as databases or application servers to a private-facing subnet.
- You can provide multiple layers of security, including security groups and network access control lists, to help control access to Amazon EC2 instances in each subnet.



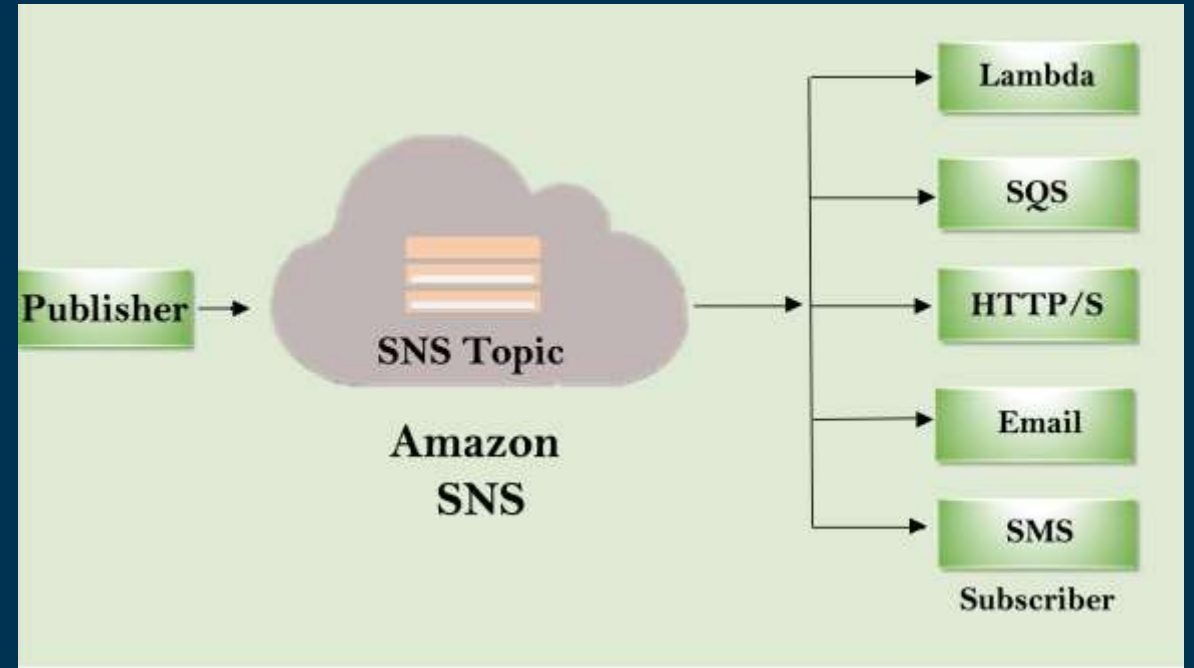
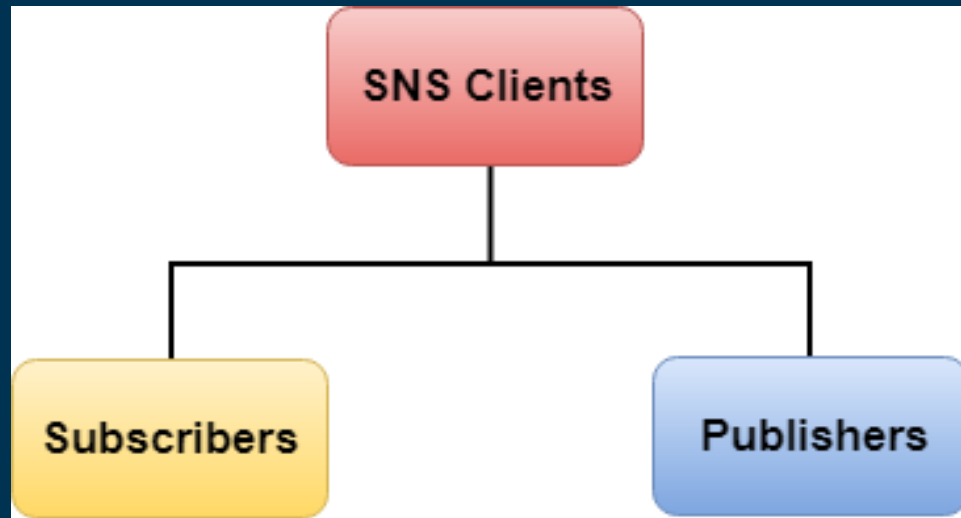
AWS SNS

Simple Notification Service

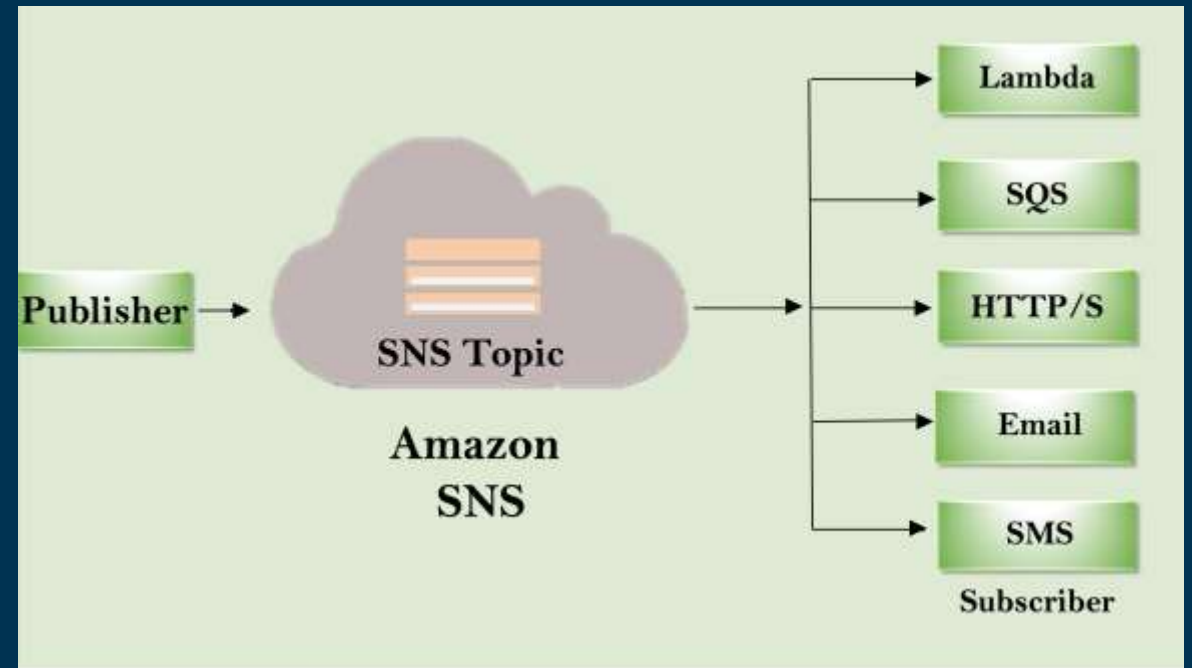
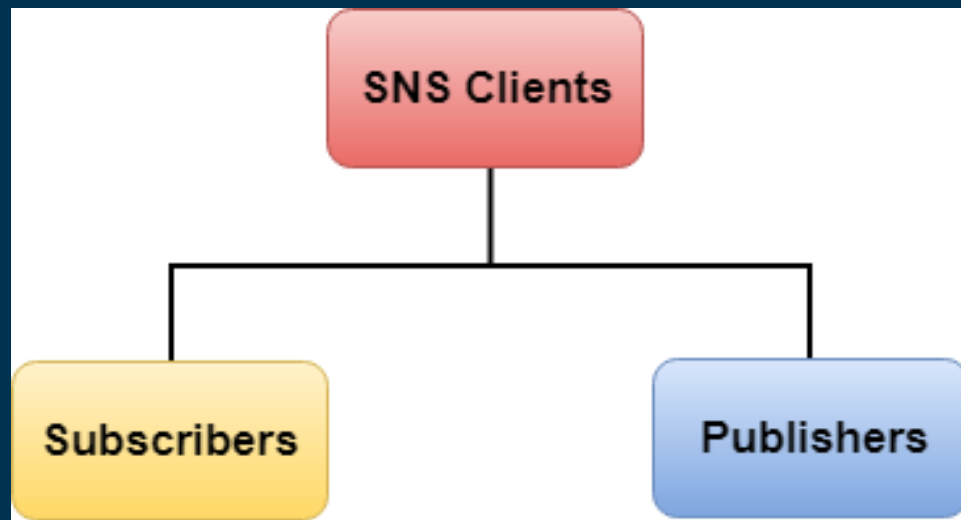


- SNS stands for Simple Notification Service.
- It is a web service which makes it easy to set up, operate, and send a notification from the cloud.
- It provides developers with the highly scalable, cost-effective, and flexible capability to publish messages from an application and sends them to other applications.
- It is a way of sending messages. When you are using AutoScaling, it triggers an SNS service which will email you that "your EC2 instance is growing".
- SNS can also send the messages to devices by sending push notifications to Apple, Google, Fire OS, and Windows devices, as well as Android devices in China with Baidu Cloud Push.
- Besides sending the push notifications to the mobile devices, Amazon SNS sends the notifications through SMS or email to an Amazon Simple Queue Service (SQS), or to an HTTP endpoint.

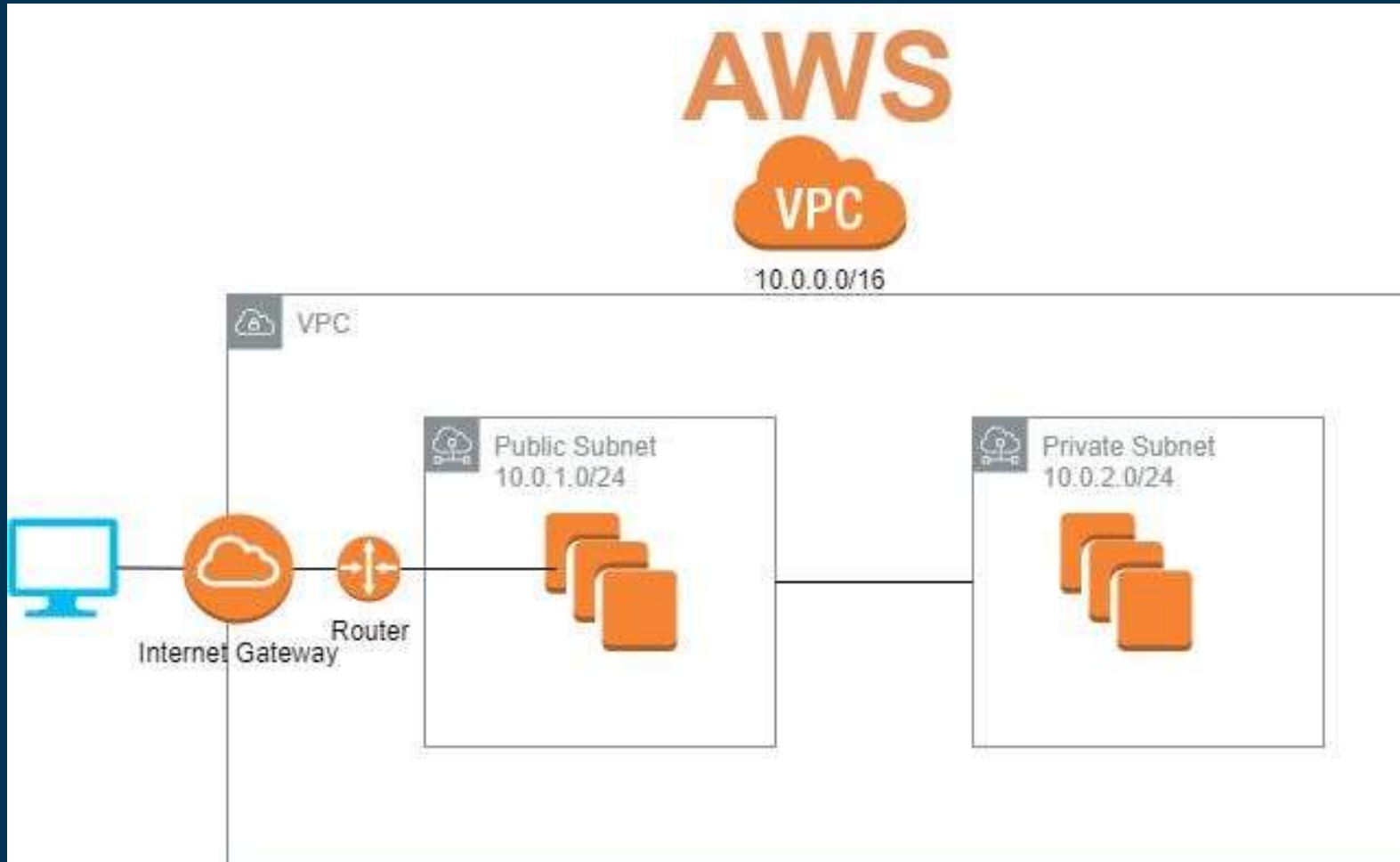
Simple Notification Service



Simple Notification Service



AWS Virtual Private Cloud

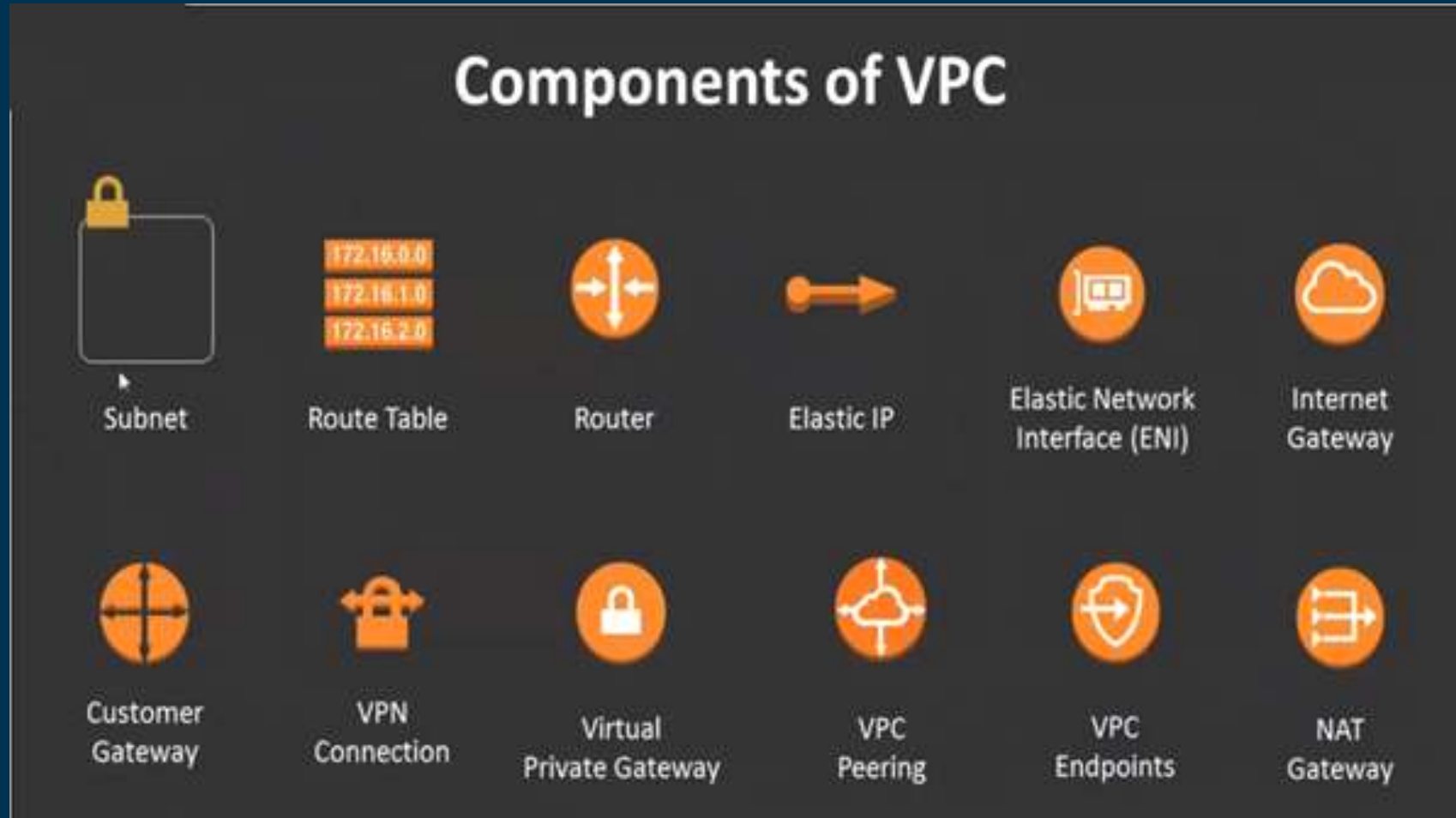


What can we do with a VPC?



- Launch instances in a subnet of your choosing. We can choose our own subnet addressing.
- We can assign custom IP address ranges in each subnet.
- We can configure route tables between subnets.
- We can create an internet gateway and attach it to our VPC.
- It provides much better security control over your AWS resources.
- We can assign security groups to individual instances.
- We also have subnet network access control lists (ACLs).

Components of VPC?



Subnets



- A **subnet** defines a range of IP addresses in your VPC. A *subnet* is a segment of an Amazon VPC's IP address range where you can launch Amazon EC2 instances.
- AWS reserves the first four IP addresses and the last IP address of every subnet for internal networking purposes. For example, a subnet defined as a /28 has 16 available IP addresses; subtract the 5 IPs needed by AWS to yield 11 IP addresses for your use within the subnet.
- Each subnet must reside entirely within one Availability Zone and cannot span zones. You can add one or more subnets in each Availability Zone.
- Majorly three types of Subnets, based on usage: **Public**, **Private** and **VPN-only**.

Reserved IP in Subnets



The first four IP addresses and the last IP address in each subnet CIDR block are not available for your use, and they cannot be assigned to a resource, such as an EC2 instance. For example, in a subnet with CIDR block 10.0.0.0/24, the following five IP addresses are reserved:

1. 10.0.0.0: Network address.
2. 10.0.0.1: Reserved by AWS for the VPC router.
3. 10.0.0.2: Reserved by AWS. The IP address of the DNS server is the base of the VPC network range plus two.
4. 10.0.0.3: Reserved by AWS for future use.
5. 10.0.0.255: Network broadcast address. We do not support broadcast in a VPC, therefore we reserve this address.

VPC and Subnets



You can launch AWS resources into a subnet that you select.

- A **private subnet** should be used for resources that won't be accessible over the Internet.
- A **public subnet** should be used for resources that will be accessed over the Internet.
- A **VPN only** subnet must talk only to Organization's VPN/services

Each subnet must reside entirely within one Availability Zone and cannot span zones.

Creating VPC



Create a Custom Amazon VPC

- Sign in to the AWS Management Console as an administrator or super user.
- Select the Amazon VPC icon to launch the Amazon VPC Dashboard.
- Create an Amazon VPC with a CIDR block equal to 10.0.0.0/16, a name tag of **My First VPC**, and default tenancy.
- You have created your first custom VPC.
- Now create a subnet under this VPC and launch a server in VPC

Security in VPC



- Security Group
- Network ACL
- Routing Tables

Working with Subnets



Working with subnets in your VPC

- Sign in to the AWS Management Console as an administrator or super user and Select your VPC under Amazon VPC Dashboard.
- Create two subnets under your VPC as: Public-Subnet - 10.0.0.0/24 in us-east-1a and Private-Subnet – 10.0.1.0/24 in us-east-1b
- Create a public server under the public subnets (one in us-east-1a)

While creating public server , make sure to enable auto assign public option during server creation.

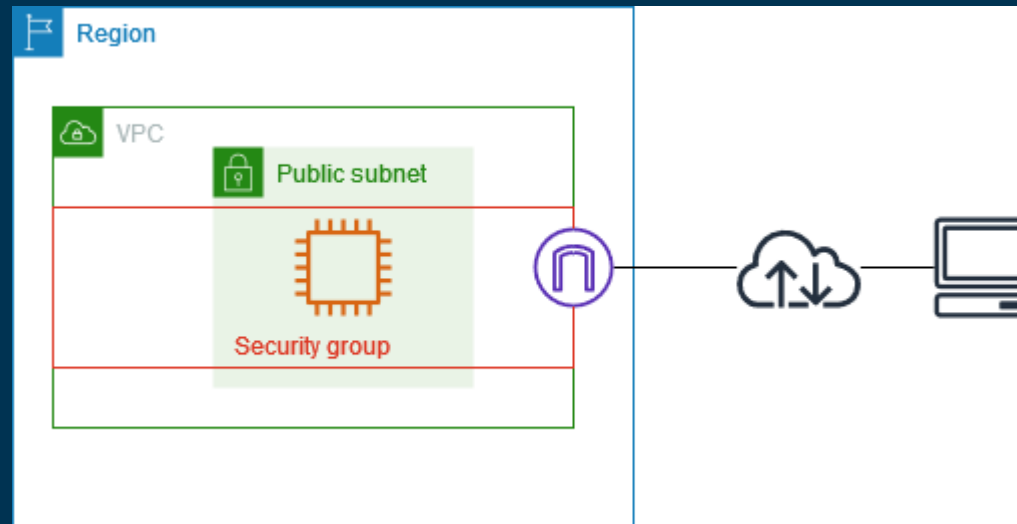
- Create a route table named public route for giving vpc access to public server externally. In the route table , associate public subnet to the public route.
- Now Assign an Internet gateway to my-vpc and edit route in public route . Now check whether public server is accessible or not.

Note: Make sure to edit routes and add internet gateway to the public route.

AWS Security Group



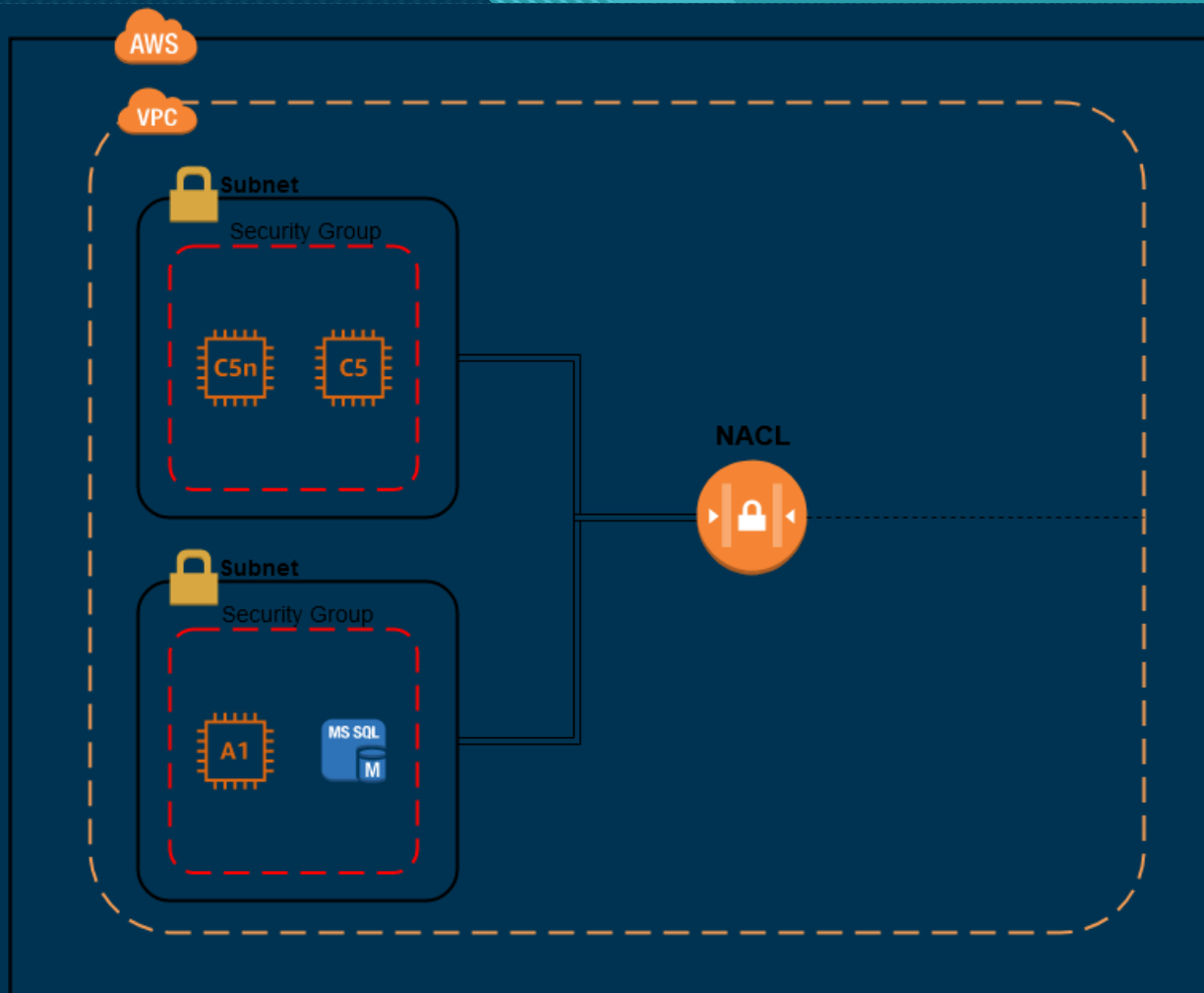
- An AWS security group acts as a virtual firewall for your EC2 instances to control incoming and outgoing traffic.
- Both inbound and outbound rules control the flow of traffic to and traffic from your instance, respectively.
- A Security group is the first defence against hackers.
- A security Group a stateful services.



AWS NACL

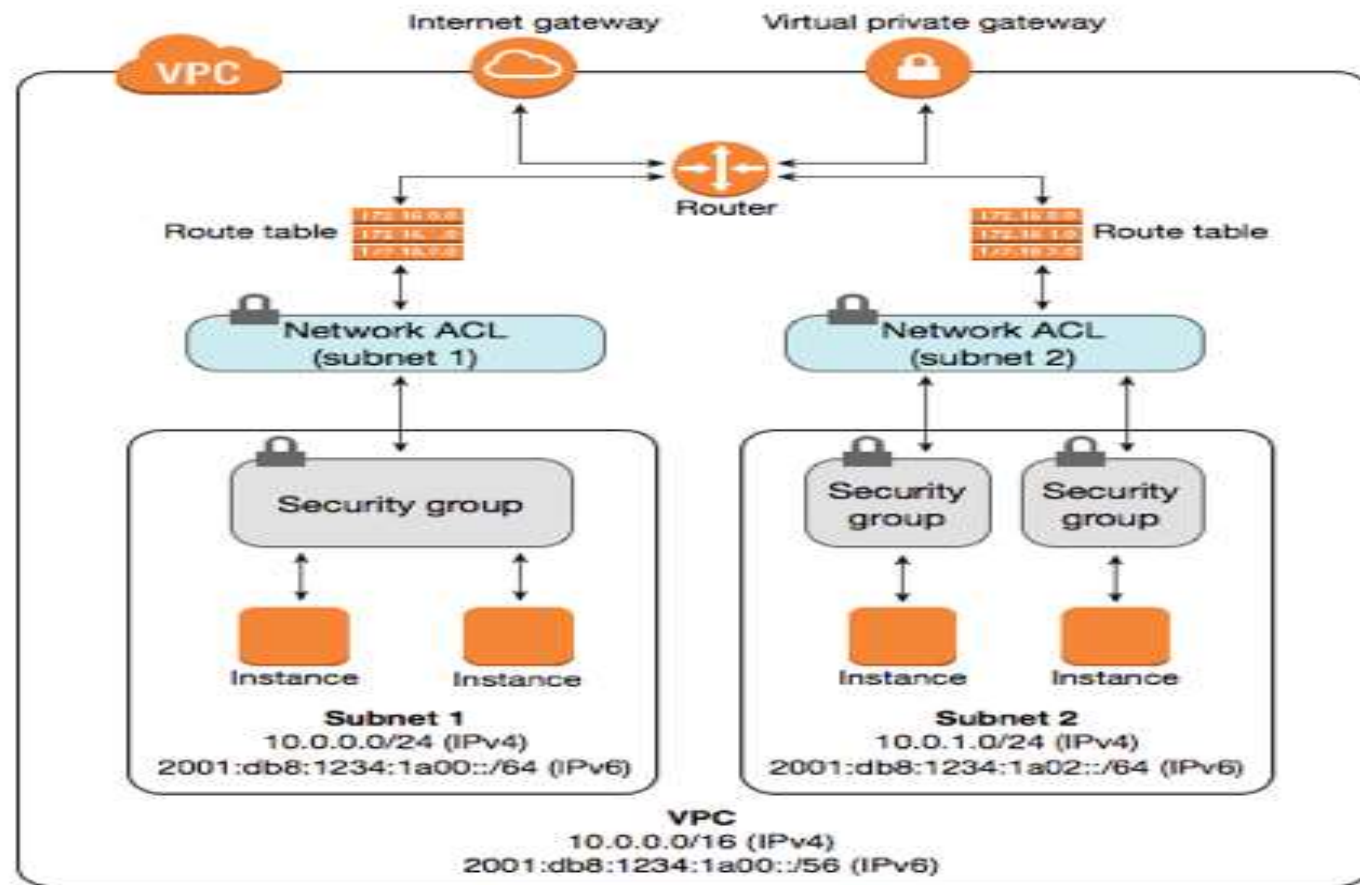
VPC

- NACL stands for Network Access Control Lists.
- It is a security layer for your VPC that controls the traffic in and out of one or more subnets.
- It is an optional layer for your VPC.
- You can set up a Network ACL similar to the security group that adds an additional layer of security to your VPC.



AWS NACL

VPC



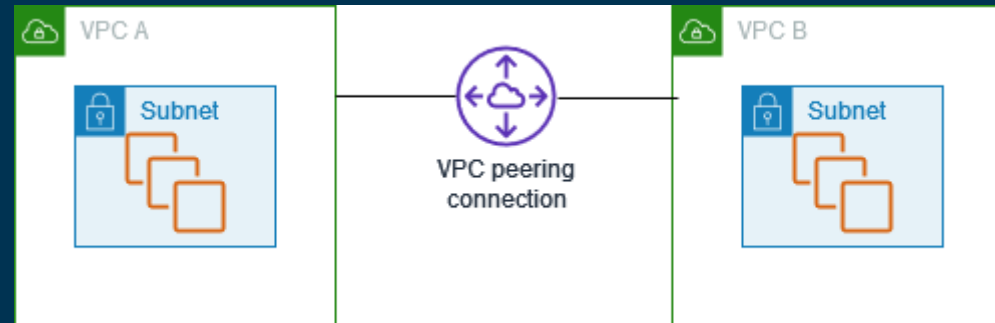


AWS VPC Peering

VPC Peering



- A VPC peering connection is a networking connection between two VPCs that enables you to route traffic between them using private IPv4 addresses or IPv6 addresses. Instances in either VPC can communicate with each other as if they are within the same network.



Types of VPC Peering



In Amazon Web Services (AWS), there are several types of VPC Peering connections based on the way peering connections are established and managed. These types include

1. Standard VPC Peering
2. Cross-account VPC Peering,
3. VPC Peering with transit gateways.

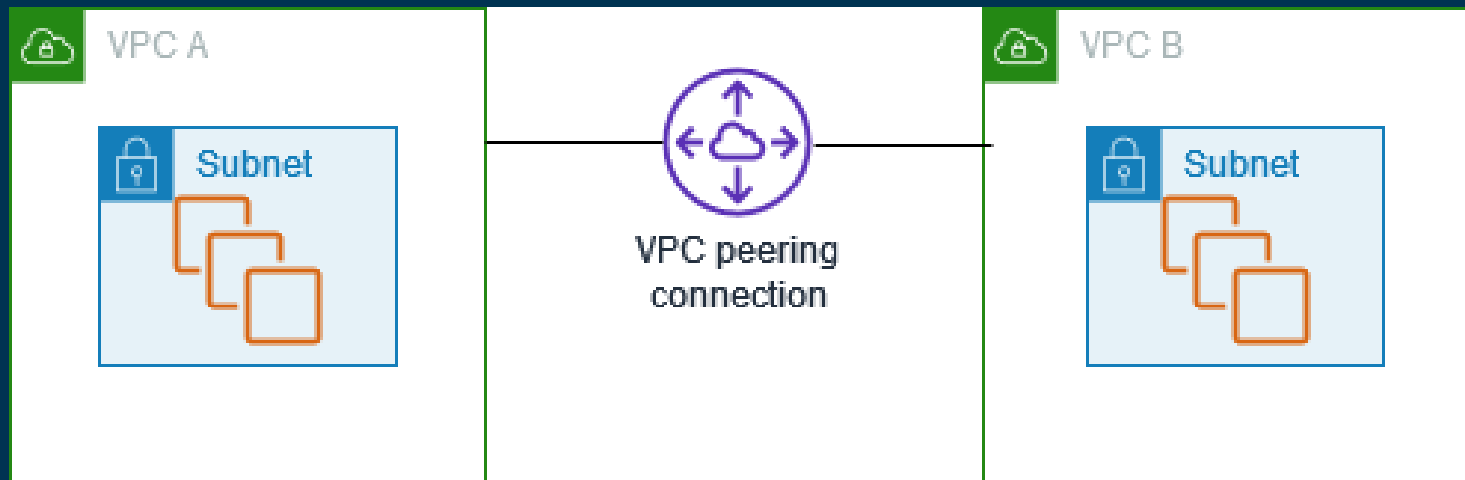
Standard VPC Peering



Within Same AWS Account: This is the standard VPC Peering where connections are established between VPCs within the same AWS account.

Single Region: VPC Peering is established within a single AWS region.

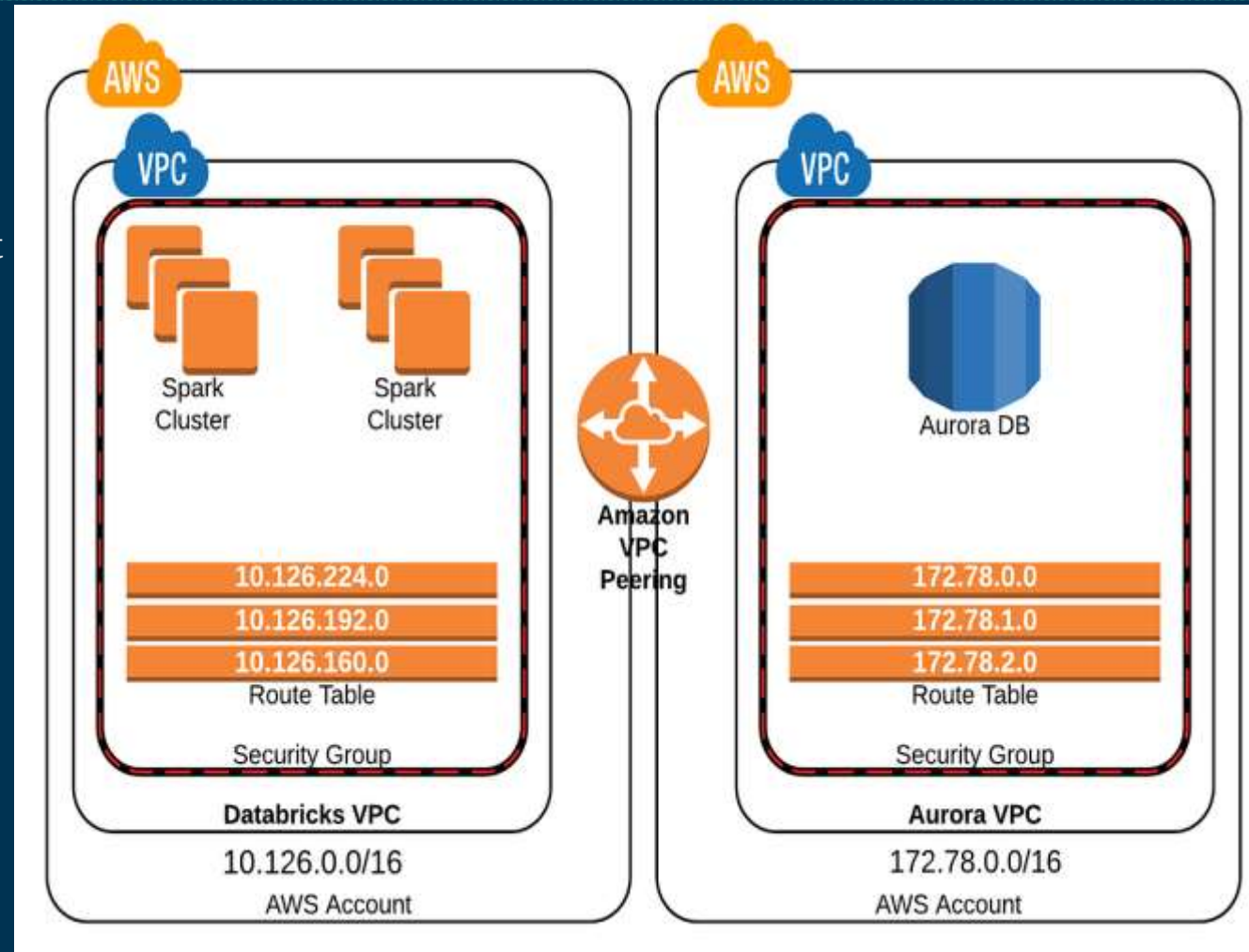
Non-Transitive: Peering connections are non-transitive, meaning that if VPC A is peered with VPC B and VPC B is peered with VPC C, VPC A cannot directly communicate with VPC C.



Cross-Account VPC Peering



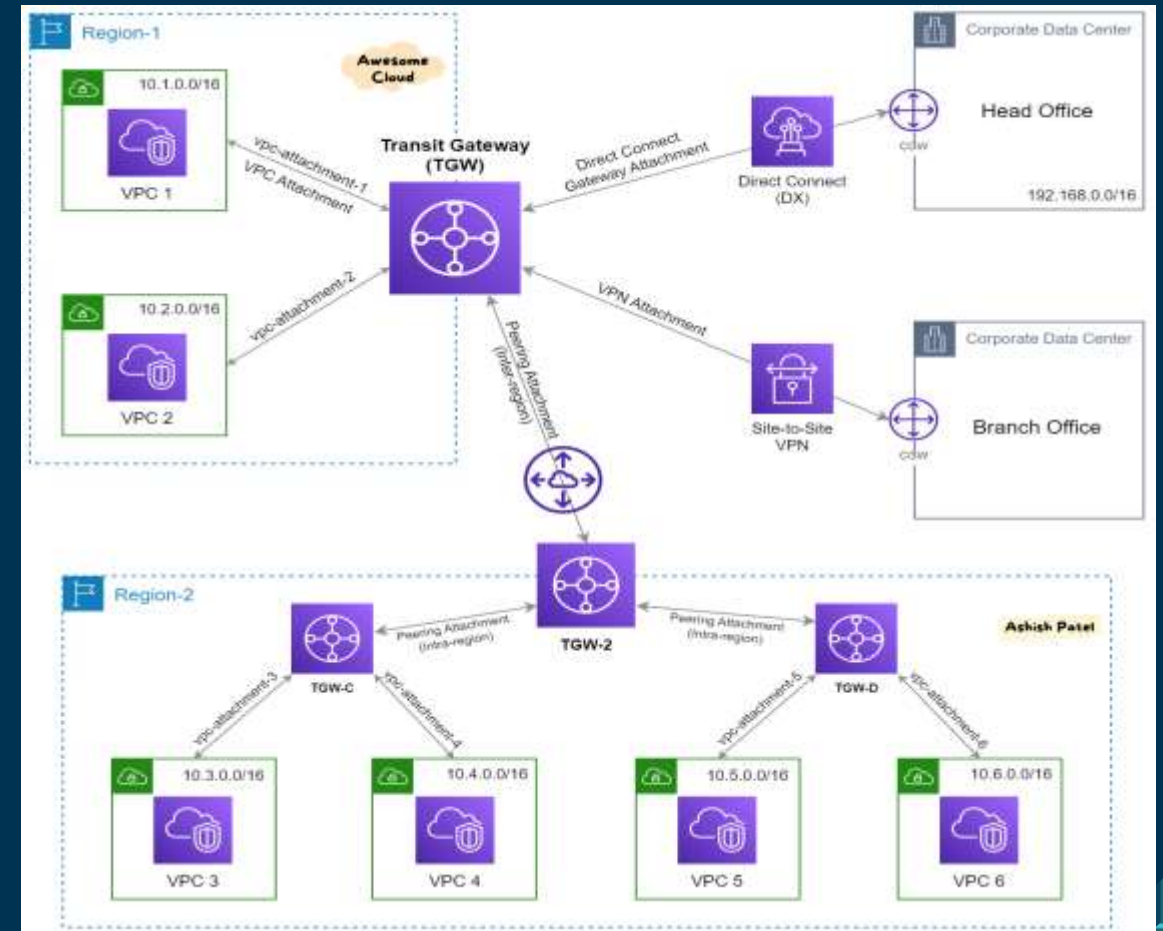
- Between Different AWS Accounts: Allows VPCs in different AWS accounts to be peered together.
- Single Region: Similar to standard VPC Peering, cross-account peering is established within a single AWS region.
- Permissions and Authorization: Requires appropriate permissions and authorization set in both AWS accounts to initiate and accept peering requests across accounts.



VPC Peering with transit gateways

VPC

- Transitive Connectivity: VPC Peering with AWS Transit Gateway allows transitive routing between multiple VPCs and remote networks connected to the Transit Gateway.
- Centralized Hub: Transit Gateway serves as a centralized hub, facilitating connections between VPCs, VPNs, and other on-premises networks.
- Cross-Region Connectivity: Transit Gateway can provide connectivity between VPCs in different AWS regions.
- Simplified Management: Allows for easier management of networking connections and reduces the need for multiple point-to-point peering connections



Lab : VPC Peering

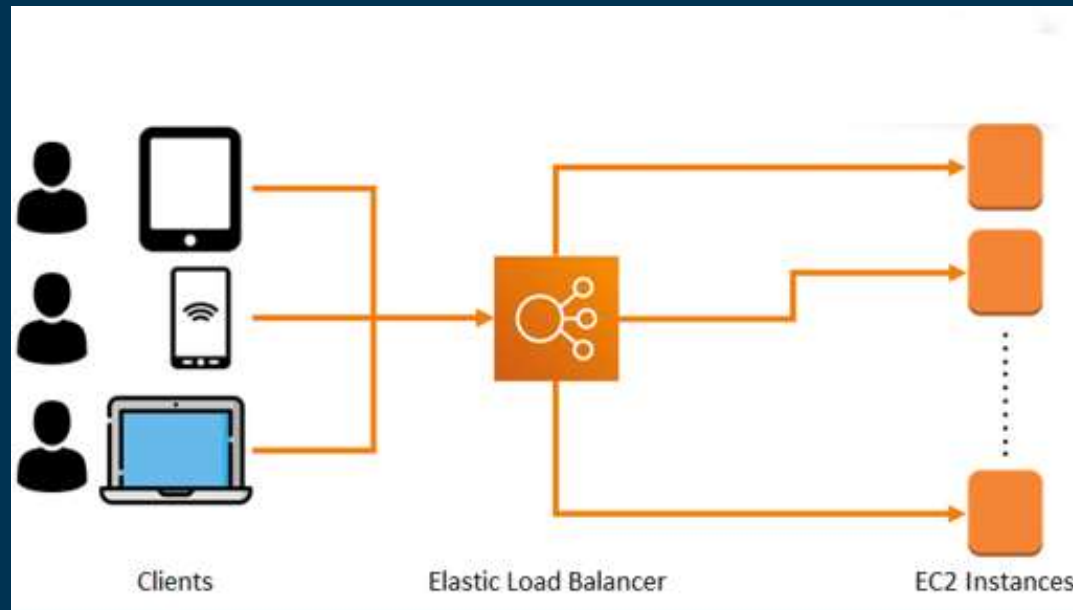


Create VPC peering among two VPCs in same region

1. Create two VPCs in Mumbai Region and two subnets in each one
2. Create two servers in each VPC
3. Verify that servers within VPC are communicating with each other, while outside VPC, same is not pingable.
4. Create VPC peering by visiting “Peering connections” under VPC dashboard
5. Select both VPCs and create VPC connection. Approve the connection.
6. Add route to route tables & verify the traffic movement between these VPCs

Amazon Load Balancer

- Amazon Load Balancer distributes network traffic across its components like Amazon EC2 instances, AWS Lambda, and containers.
- Supports health checks to detect unhealthy Amazon EC2 instances
- Supports the routing and load balancing of HTTP, HTTPS, and TCP traffic to Amazon EC2 instances
- Works within Region and across Availability zones, which means it can divert traffic to servers in different AZs, but within same Region.



Benefits of Using ELB



- It is highly available and distributes website traffic across multiple targets
- It provides high security through user authentication and SSL/TLS decryption features
- It can handle drastic changes in website traffic without human intervention
- It helps improve the visibility of your applications through continuous monitoring and auditing
- It supports hybrid load balancing, which can help when migrating resources to the cloud



Types of AWS ELB

Here are mainly four types of load balancers available on AWS.

- **Classic Load Balancer :** This is the most basic form of the load balancer, that was used initially for classic EC2 instances. It is operated on connection level as well as request level. The main drawback of this type of load balancer is, it does not support features like host-based routing or path-based routing.
- **Network Load Balancer :** Network Load Balancer works at layer 4 of the OSI Model(connection level). Its mainly used for load balancing TCP traffic. This Load Balancer can handle millions of traffic and is best suited for maintaining low latencies
- **Application Load Balancer :** This type of Load Balancer is used when there are HTTP and HTTPS traffic routing.. This load balancer works at the Application layer of the OSI Model. It provides advanced routing features such as host-based and path-based routing. It also works well with containers and microservices

Lab : Working with ELB



1. Create two AmazonLinux servers in different-2 Azs (us-east-1a &us-east1b)
2. Make sure to add html script user-data for post-verifying on both servers.
3. Create a security for alb---ec2 for access of traffic inside ec2 instances from our alb giving port 22 and 80 access to public for now and configure them on both servers us-east-1a and 1b while creating the instances.
4. After creating both servers, create the application load balancer and while creating the LB , create a separate security group for ALB to give access to external users to our load balancer.
5. Create a target group adding both instances to that target group.
6. After adding a target group , click on create load balancer and wait for sometime.
7. Now add user—ec2 securitygroup to alb—ec2instances security group to disallow the external users to access our internal instances behind the load balancers directly.

Amazon CloudWatch

- Amazon CloudWatch is an AWS monitoring and management service which is designed for the purpose of maintaining the services and resources which are used.
- This services is designed for developers, site reliability engineers, IT managers, and system operators to make their life easier.



The background of the slide is split diagonally from the bottom-left to the top-right. The upper-left portion is a solid light blue. The lower-right portion is a dark blue with a fine, repeating grid pattern. A diagonal band of thin, horizontal light blue lines separates the two main background areas.

Amazon CloudFormation

CloudFormation

- AWS CloudFormation is a service that helps you model and set up your Amazon Web Services resources so that you can spend less time managing those resources and more time focusing on your applications that run in AWS. You create a template that describes all the AWS resources that you want (like Amazon EC2 instances or Amazon RDS DB instances), and AWS CloudFormation takes care of provisioning and configuring those resources for you.

Benefits of AWS CloudFormation

- Simplify Infrastructure Management
- Quickly Replicate Your Infrastructure
- Easily Control and Track Changes to Your Infrastructure

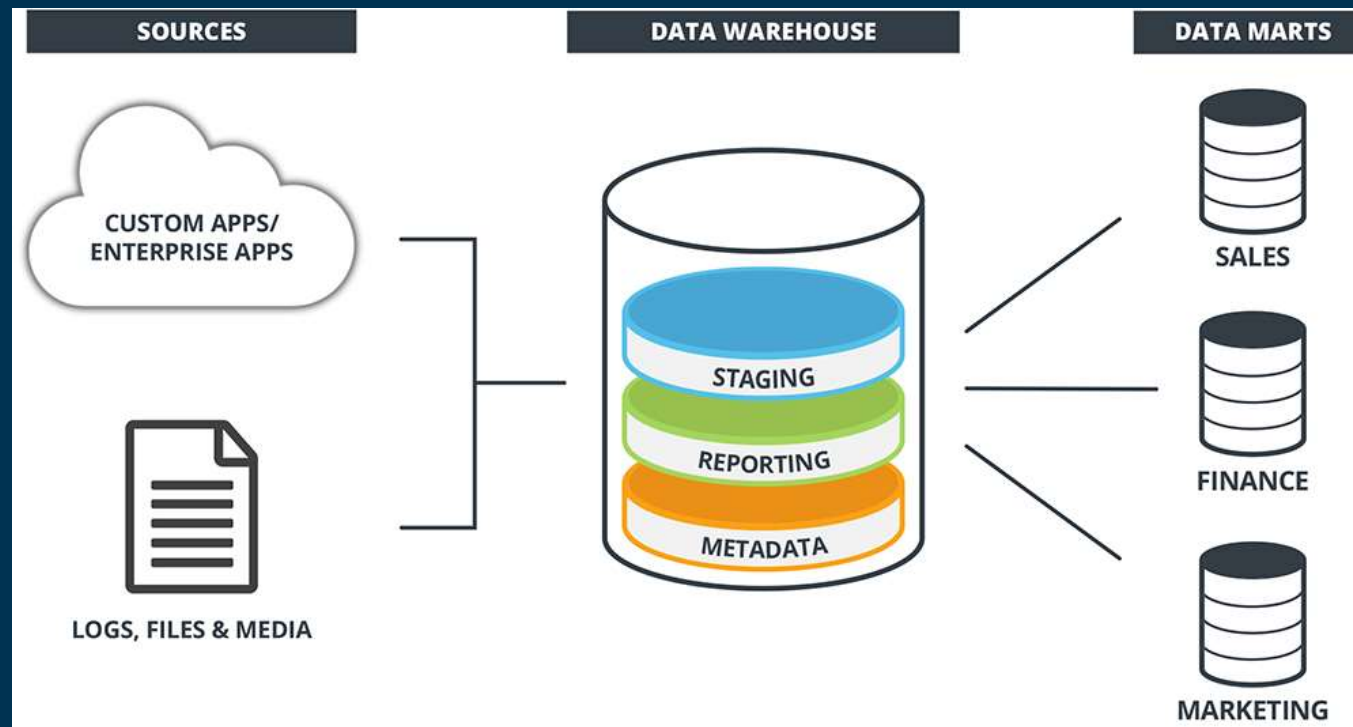
The background of the slide is split diagonally from the bottom-left to the top-right. The upper-left portion is a solid light blue. The lower-right portion is a dark blue with a fine, repeating diagonal line pattern. A thick, solid dark blue diagonal band runs from the bottom-left towards the top-right, separating the two background sections.

Amazon Redshift

Data Warehouse

A data warehouse is a central repository for storing, managing, and analyzing a large amount of structured and sometimes unstructured data from various sources within an organization.

It's designed to support business decision-making processes by providing a consolidated and organized view of data across the enterprise.



Why data warehouses are used

1. **Centralization of Data:** Data warehouses gather information from disparate sources within an organization, including databases, spreadsheets, CRM systems, ERP systems, etc. It consolidates this data into a single, unified repository, allowing for easier access and analysis.
2. **Data Integration:** Different systems often use different formats and structures for storing data. Data warehouses integrate data from various sources, transforming it into a common format, making it easier to analyze and query.
3. **Historical Data Storage:** Data warehouses store historical data over time. This historical perspective enables trend analysis, pattern recognition, and forecasting, which can be crucial for making informed business decisions.
4. **Improved Data Quality:** Through data cleaning, transformation, and normalization processes, data warehouses help maintain data accuracy and consistency, improving overall data quality.
5. **Support for Decision-Making:** Businesses can use data warehouses to perform complex queries and analysis. Decision-makers can derive insights and create reports or dashboards that aid in strategic planning, identifying trends, and making data-driven decisions.
6. **Scalability and Performance:** Data warehouses are designed to handle large volumes of data and complex queries efficiently. They are optimized for query performance, enabling faster access to information compared to traditional databases.
7. **Business Intelligence (BI) and Analytics:** Data warehouses serve as a foundation for business intelligence and analytics tools. By providing a structured and reliable data source, these tools can extract meaningful insights, generate reports, visualize data, and conduct predictive analysis, among other things.

Amazon Redshift

- Amazon Redshift is a fully managed, cloud-based data warehousing service offered by Amazon Web Services (AWS).
- It's designed for large-scale data analytics, enabling users to efficiently query and analyze vast amounts of structured data using standard SQL queries.
- Redshift is known for its scalability, performance, and cost-effectiveness in handling complex analytical workloads.
- It is fully managed, relational database management service based on the PostgreSQL engine.
- It can handle petabyte-scale data while offering lightning-fast querying performance.

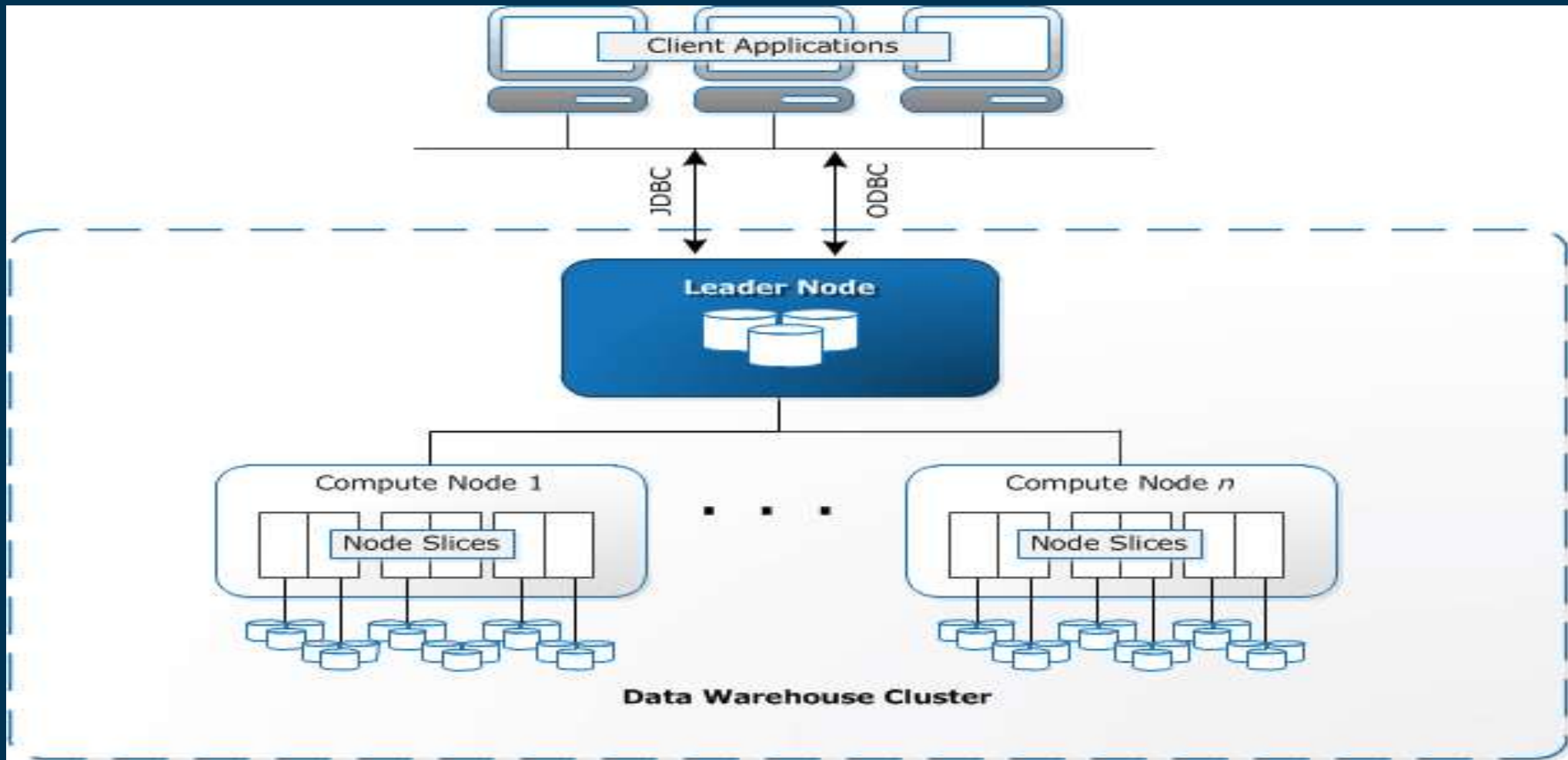
Features of AWS Redshift

1. **Columnar Storage:** Redshift organizes data in a columnar storage format, which enhances query performance and reduces I/O overhead by only reading the columns necessary for a particular query.
2. **Massive Parallel Processing (MPP):** It uses MPP architecture to parallelize and distribute queries across multiple nodes for faster execution. This allows for quick analysis of large datasets by leveraging the power of clustered computing.
3. **Scalability:** Redshift allows easy scaling by adding or removing nodes to meet changing workload demands. It offers both dense compute and dense storage node types to cater to different requirements.
4. **Integration with Data Lake:** Redshift integrates seamlessly with AWS services like Amazon S3, allowing users to query and analyze data across a data lake and data warehouse using Redshift Spectrum. This enables businesses to run queries on unstructured or semi-structured data stored in S3 without needing to load it into Redshift.
5. **Ease of Management:** As a managed service, AWS handles administrative tasks such as provisioning, configuring, monitoring, and patching, allowing users to focus more on analytics and less on infrastructure management.
6. **Analytics and Business Intelligence (BI) Integration:** Redshift integrates with various BI tools, visualization platforms, and ETL (Extract, Transform, Load) tools, enabling users to easily connect their preferred analytics and reporting tools for data analysis and visualization.
7. **Cost-Effective:** Redshift offers a pay-as-you-go pricing model, allowing users to pay only for the resources they use, making it cost-effective compared to traditional on-premises data warehouses.

Clusters and Nodes

- The key component of an Amazon Redshift data warehouse is a cluster.
- A cluster is composed of a leader node and one or more compute nodes.
- The client application interacts directly only with the leader node, and the compute nodes are transparent to external applications.
- Amazon Redshift currently has support for six different node types and each has a different mix of CPU, memory, and storage.
- The six node types are grouped into two categories: **Dense Compute and Dense Storage**.
- The Dense Compute node types support clusters up to 326TB using fast SSDs, while the Dense Storage nodes support clusters up to 2PB using large magnetic disks.

Amazon Redshift



Amazon Redshift cluster architecture

- Each cluster contains one or more databases. User data for each table is distributed across the compute nodes. Your application or SQL client communicates with Amazon Redshift using standard JDBC or ODBC connections with the leader node, which in turn coordinates query execution with the compute nodes. Your application does not interact directly with the compute nodes.
- The disk storage for a compute node is divided into a number of slices. The number of slices per node depends on the node size of the cluster and typically varies between 2 and 16. The nodes all participate in parallel query execution.
- Amazon Redshift allows you to resize a cluster to add storage and compute capacity over time as your needs evolve. Whenever you perform a resize operation, Amazon Redshift will create a new cluster and migrate data from the old cluster to the new one. During a resize operation, the database will become read-only until the operation is finished.

Lab : Setup Amazon Redshift Cluster

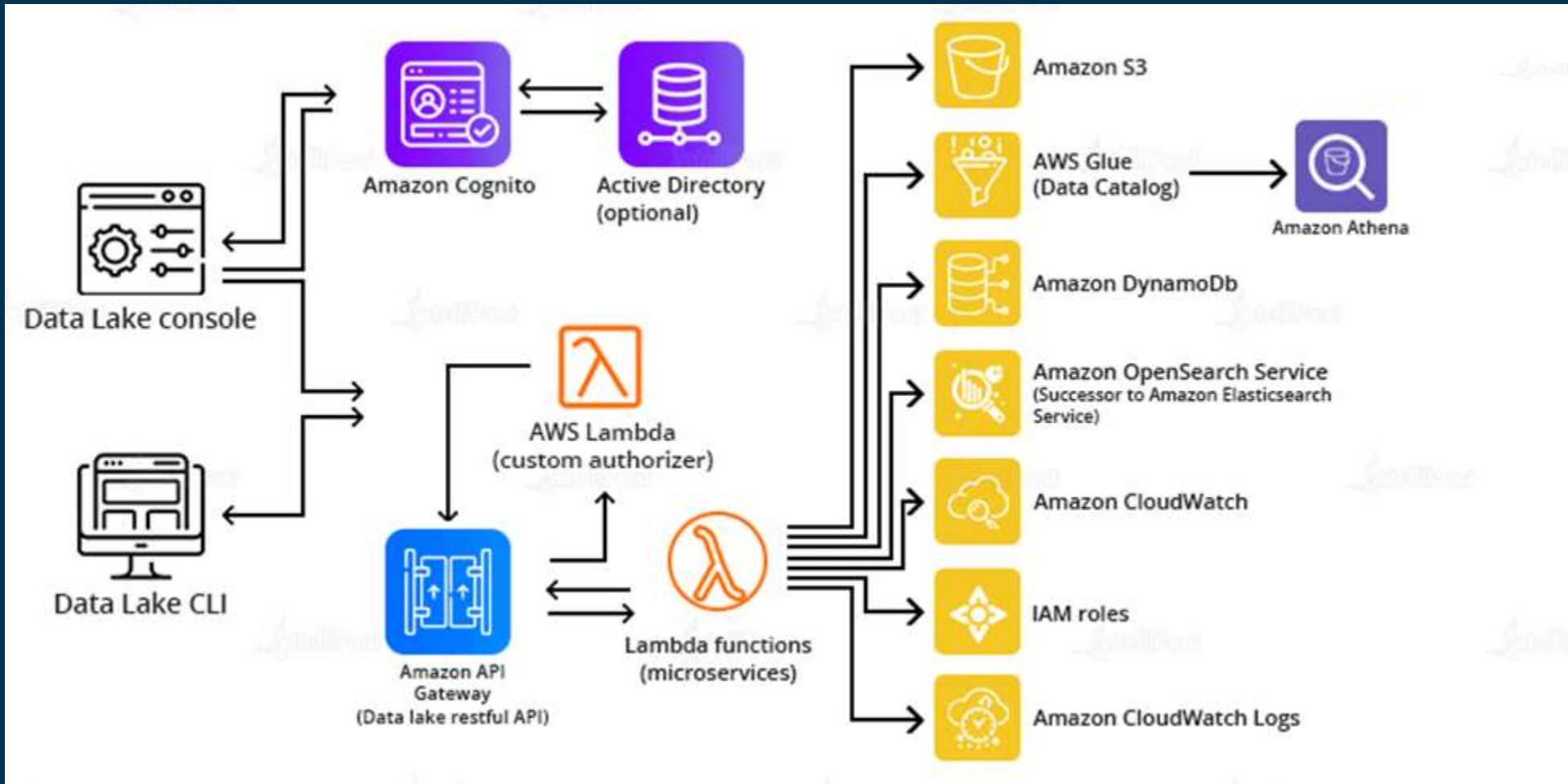
AWS Lake Formation

- AWS Lake Formation is a service provided by Amazon Web Services (AWS) that simplifies and accelerates the process of building, securing, and managing data lakes in the cloud.
- A data lake is a centralized repository that allows you to store structured, semi-structured, and unstructured data at scale.
- It enables organizations to analyze large volumes of diverse data types for analytics, machine learning, and other data-driven tasks.

Features & Functionalities of AKF

1. **Data Ingestion:** Lake Formation facilitates the ingestion of data from various sources into the data lake. It supports a range of data formats and sources, including databases, streaming data sources, on-premises storage, Amazon S3, and more. This allows for seamless integration and consolidation of data.
2. **Data Catalog:** Lake Formation provides a central metadata catalog that organizes and indexes metadata information about the data stored in the lake. It allows users to easily discover, search, and understand available datasets, their schemas, and properties.
3. **Data Security and Governance:** Security and governance are integral aspects of Lake Formation. It offers fine-grained access control, allowing users to define and manage permissions for different datasets and users. It also integrates with AWS Identity and Access Management (IAM) for authentication and authorization, as well as supports encryption for data at rest and in transit.
4. **Data Cleaning and Transformation:** Lake Formation includes built-in capabilities for data cleaning, transformation, and normalization. Users can define and apply transformations to prepare the data for analysis or downstream applications.
5. **Data Access and Querying:** Once data is cataloged and organized, users can query and analyze it using various AWS analytics and machine learning services such as Amazon Athena, Amazon Redshift Spectrum, Amazon EMR, and AWS Glue ETL jobs.
6. **Automated Workflows:** Lake Formation allows the creation of automated workflows for data ingestion, transformation, and management through AWS Glue. This helps in streamlining and orchestrating complex data pipelines.
7. **Integration with AWS Services:** Lake Formation integrates seamlessly with other AWS services like Amazon S3, AWS Glue, AWS IAM, and various analytics and machine learning services, providing a comprehensive ecosystem for building and utilizing data lakes.

AWS Data Lake Architecture



AWS Glue

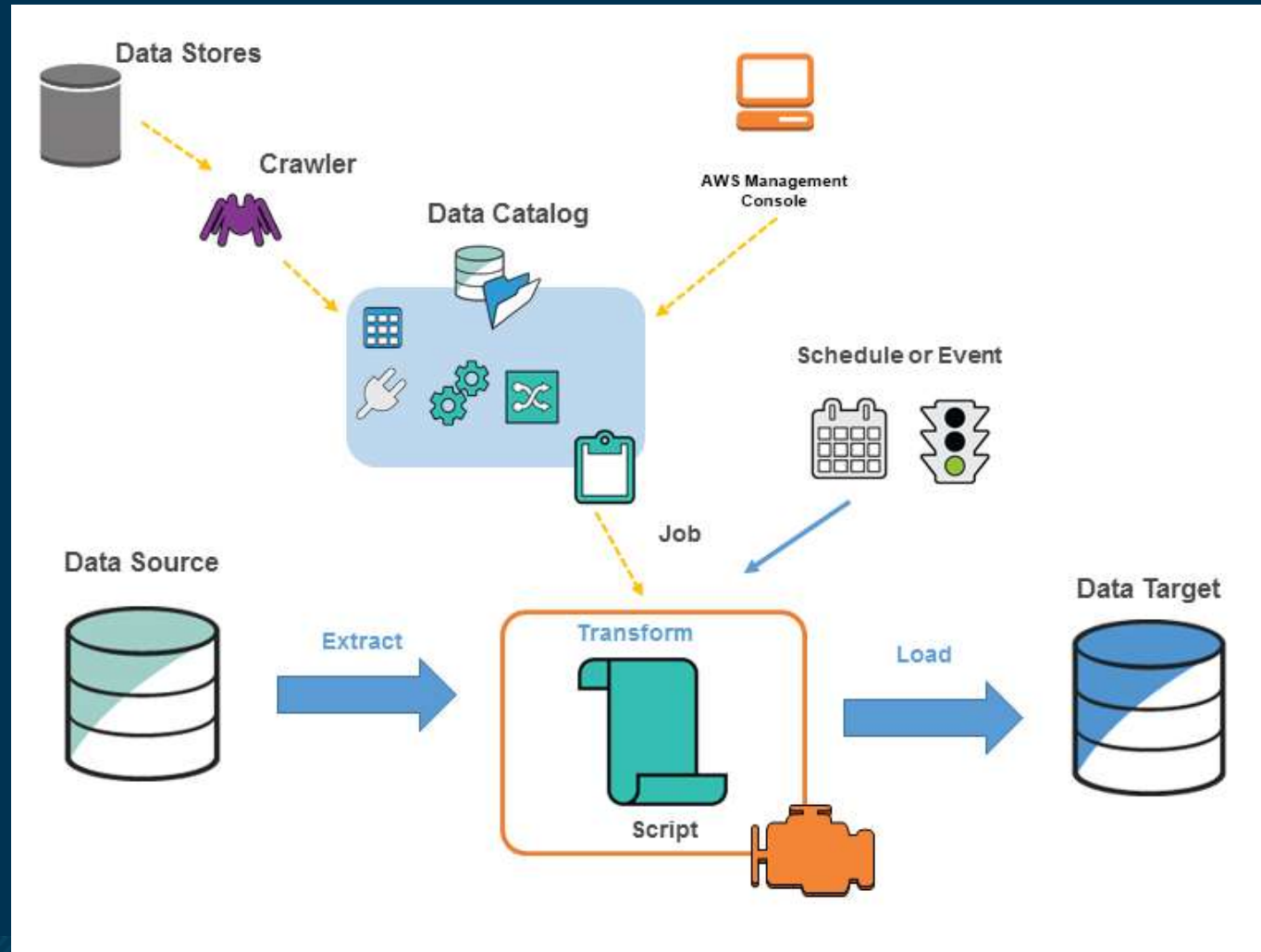
AWS Glue is simply a serverless ETL tool. ETL refers to three (3) processes that are commonly needed in most Data Analytics / Machine Learning processes: Extraction, Transformation, Loading. Extracting data from a source, transforming it in the right way for applications, and then loading it back to the data warehouse.

Amazon Glue consists of three components namely, the AWS Glue Data Catalog, an ETL engine that creates Python or Scala code automatically, and a configurable scheduler that manages dependence resolutions, task monitoring, and restarts.

AWS Glue Component

- **AWS Glue Data Catalog**
- **Classifier**
- **Connection**
- **Crawler**
- **Database**
- **Data Store**
- **Data Source**
- **Data Target**
- **Transform**
- **Development Endpoint**
- **Dynamic Frame**
- **Job**
- **Trigger**
- **Notebook Server**
- **Script**
- **Table**

AWS Glue Architecture



Lab : AWS Glue

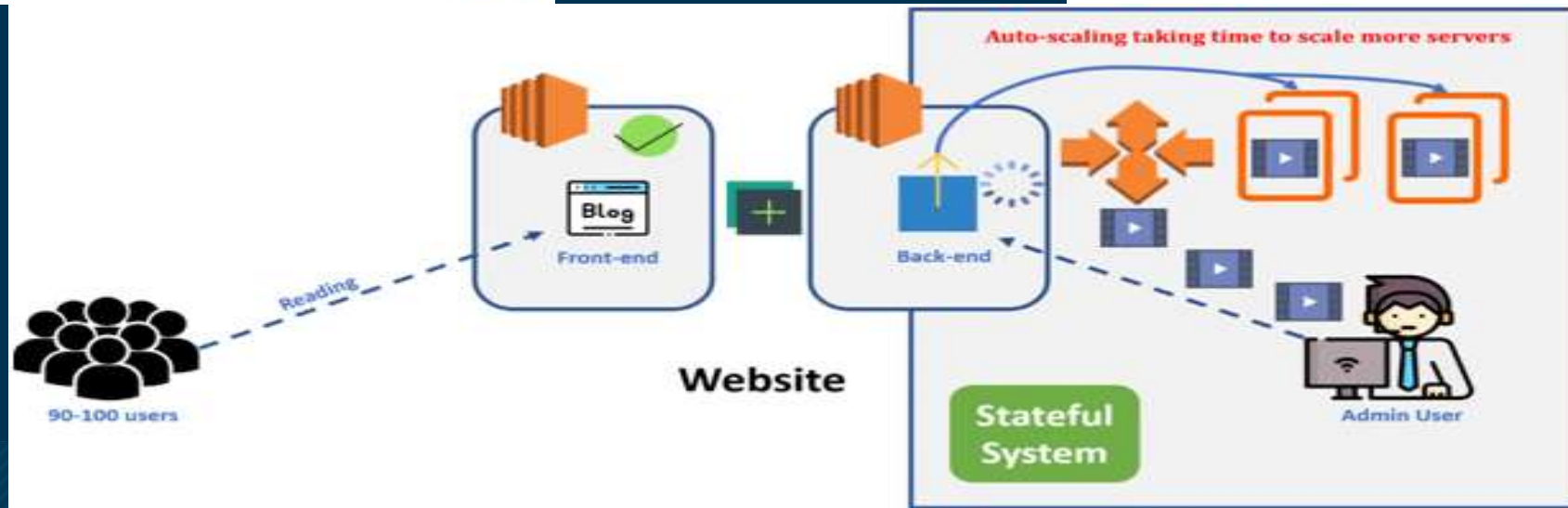
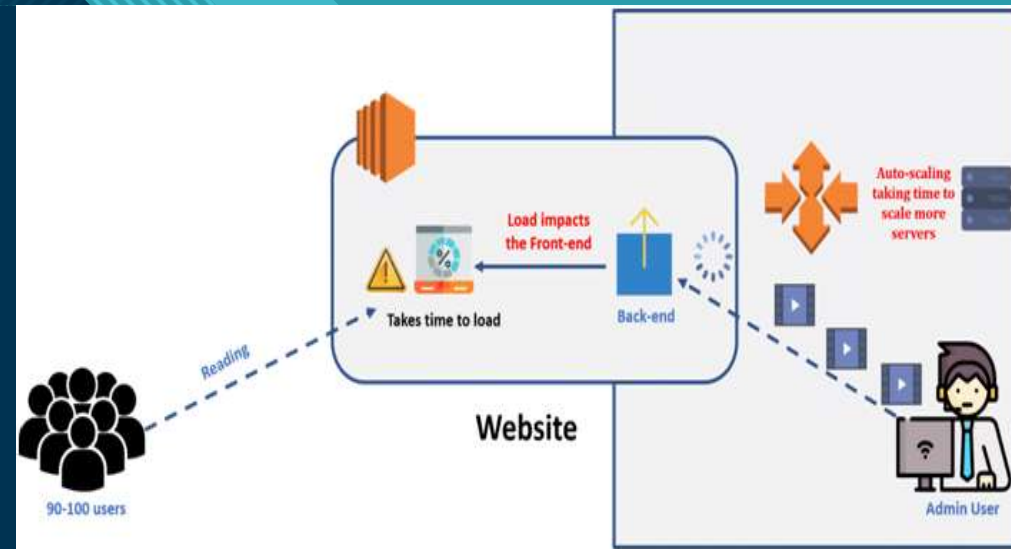
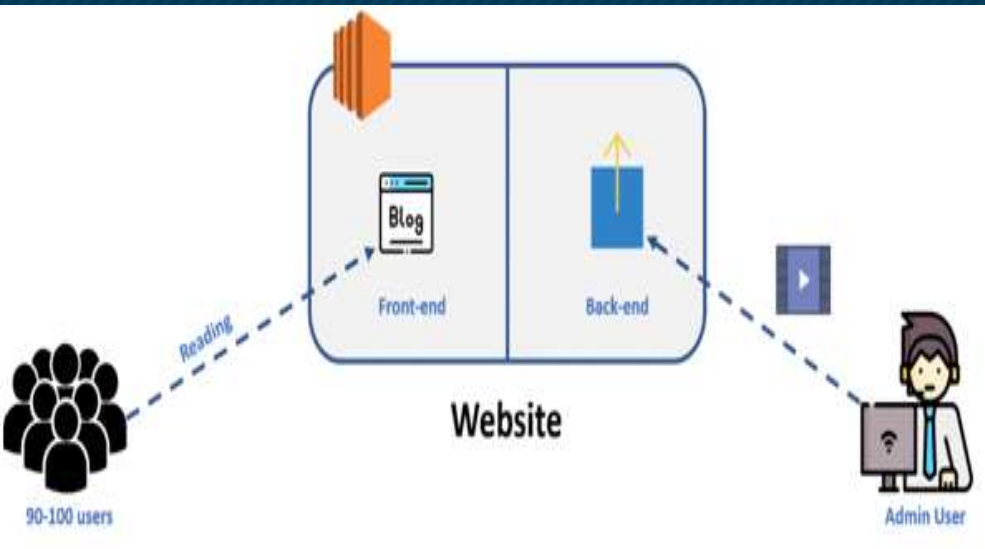
AWS Lambda



Amazon EC2



Need for AWS Lambda



AWS Lambda

- AWS Lambda is a compute service that lets you run code without provisioning or managing servers.
- The Lambda functions can perform any kind of computing task, from serving web pages and processing streams of data to calling APIs and integrating with other AWS services.
- Lambda runs your code on a high-availability compute infrastructure and performs all of the administration of the compute resources, including server and operating system maintenance, capacity provisioning and automatic scaling, and logging.
- Once you upload the source code file into AWS lambda in the form of ZIP file then AWS lambda will automatically run the code without you provision the servers and also it will automatically scaling your functions up or down based on demand.
- AWS lambda are mostly used for the event-driven application for the data processing Amazon S3 buckets, or responding to HTTP requests.

Examples of AWS Lambdas:

- Process data from Amazon S3 buckets.
- Respond to HTTP requests.
- Build serverless applications.

Benefits of AWS Lambda



Serverless Architecture



No VM Required



Code Independently



Monitor the performance

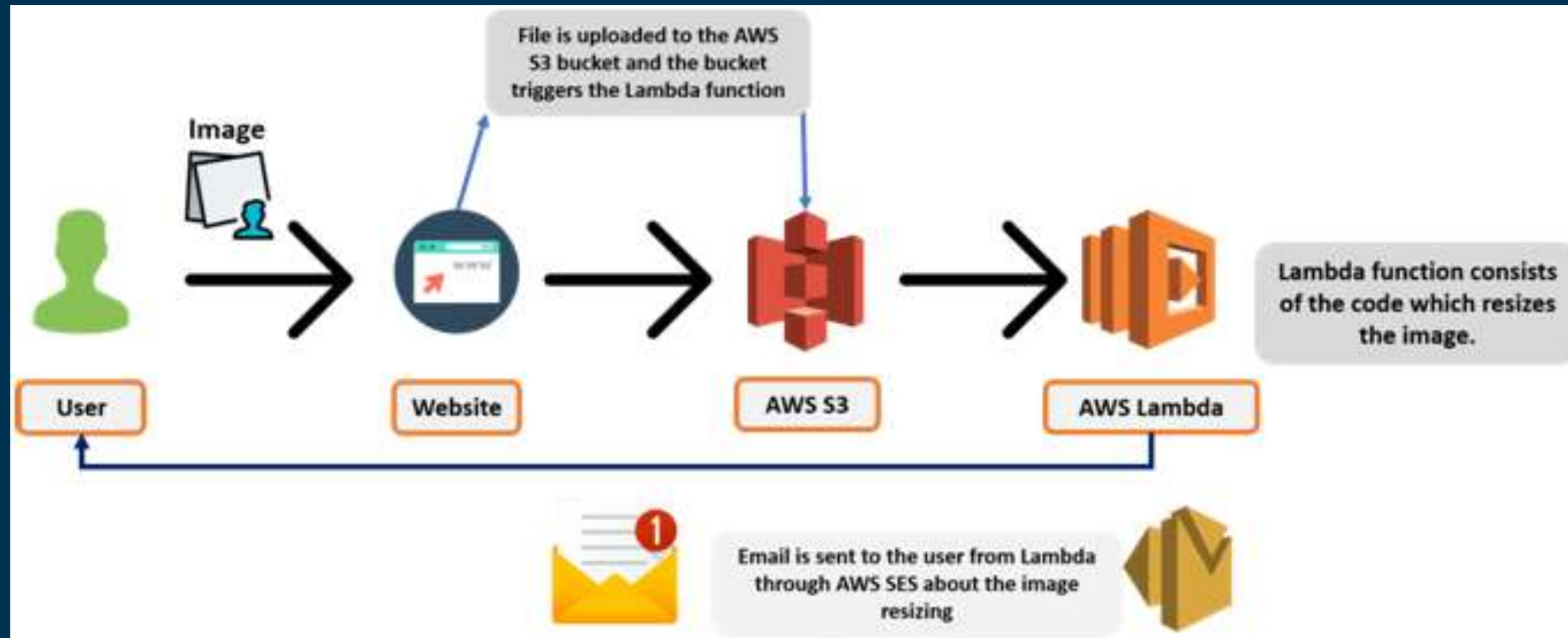


Pay as you go

Limitations of AWS Lambda

- The maximum execution duration per request is set to 300 seconds (15 minutes)
- In the case of hardware, the maximum disk space provided is 512 MB for the runtime environment, which is very less
- Its memory volume varies from 128 MB to 1,536 MB
- The event request body cannot exceed more than 128 KB
- Lambda functions write their logs only to CloudWatch, which is the only tool available in order to monitor or troubleshoot our functions

Lab : AWS Lambda



AWS Auto Scaling

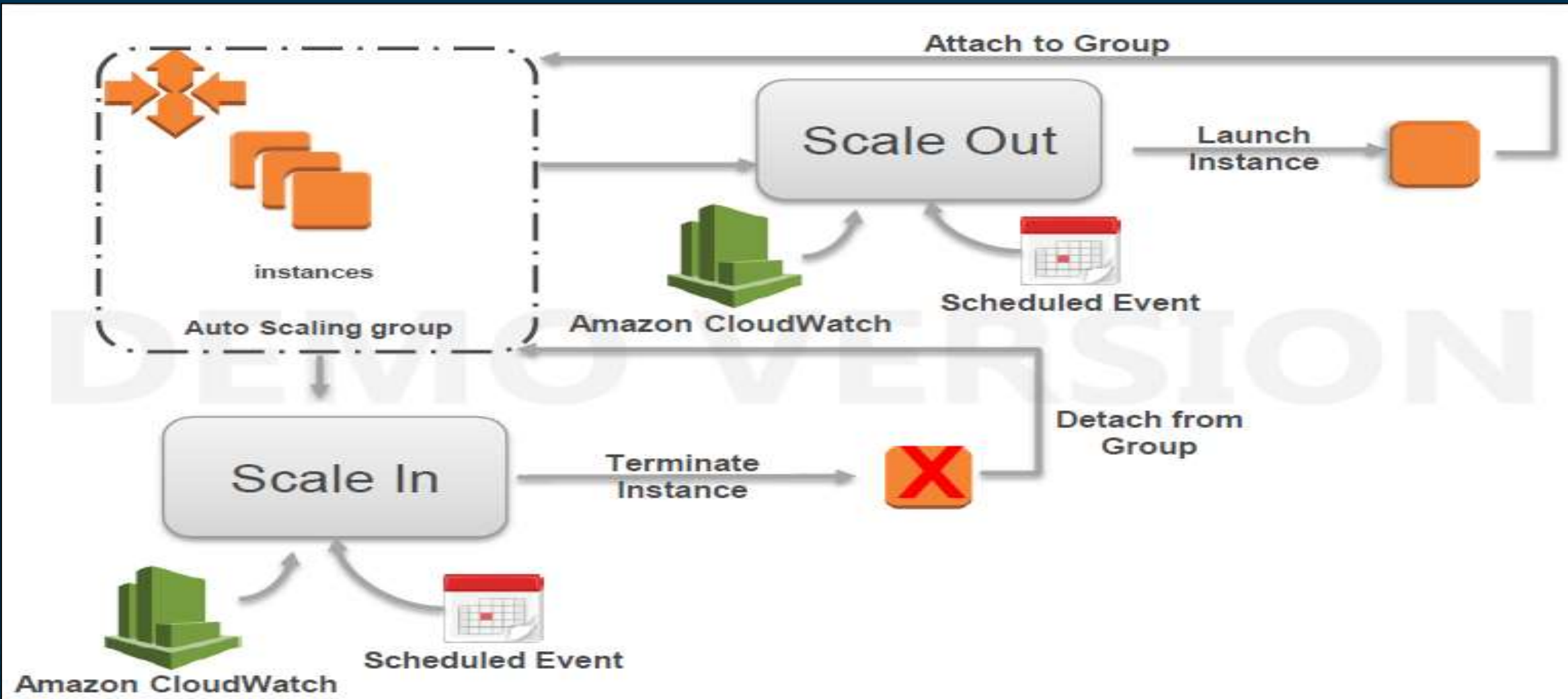
What do you understand by “Scaling”?

AWS Auto Scaling

AWS Auto Scaling

What is the difference between Scale-in and Scale-out?

AWS Auto Scaling

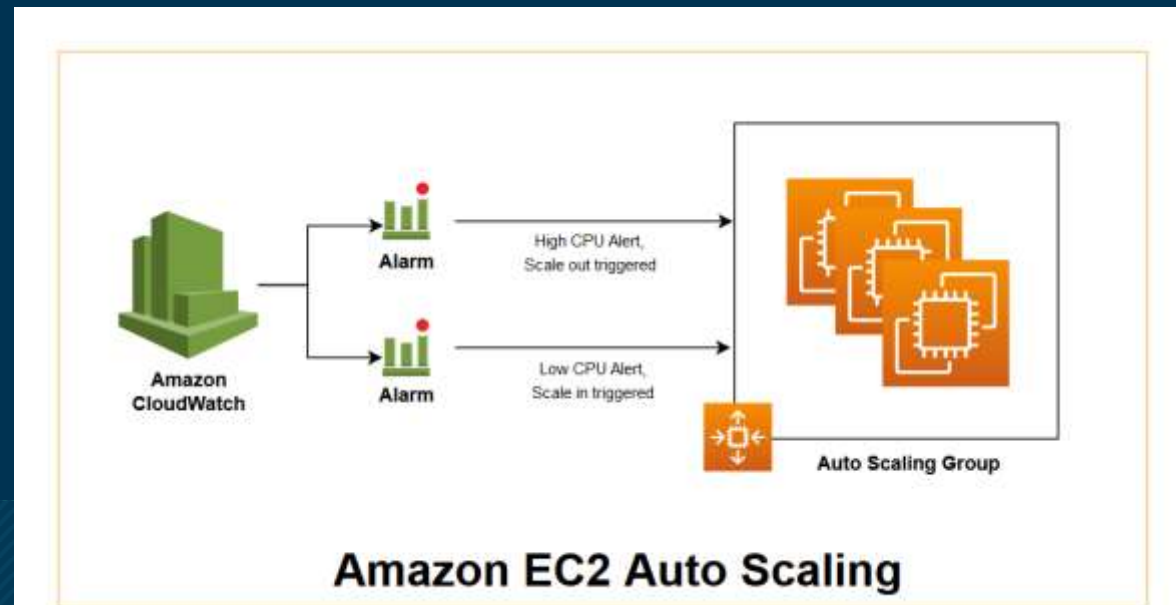


AWS Auto Scaling

What is Auto-Scaling and Why required?

AWS Auto Scaling

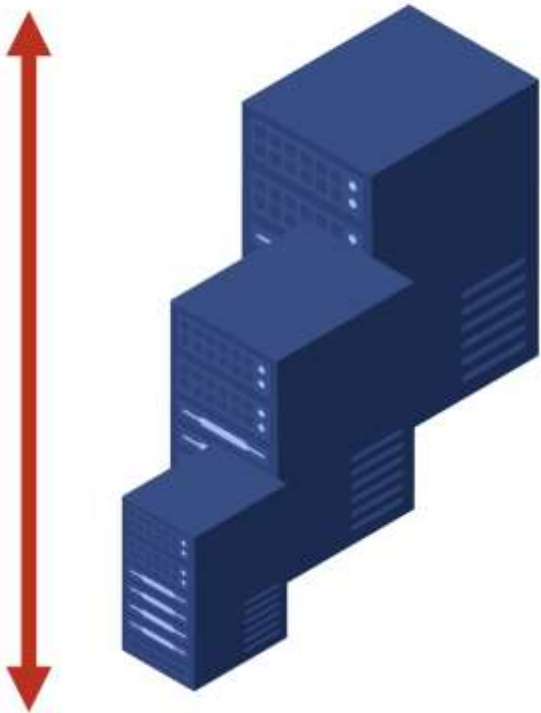
- AWS Auto Scaling is a service provided by Amazon Web Services (AWS) that allows users to automatically adjust the capacity of their Amazon EC2 (Elastic Compute Cloud) instances or other resources to maintain performance and optimize costs.
- Auto Scaling helps in ensuring that the number of instances or resources running meets the demands of your applications without over-provisioning or under-provisioning.
- It can automatically add instances when demand increases and remove instances when demand decreases, based on predefined conditions and scaling policies.



Types of Scaling

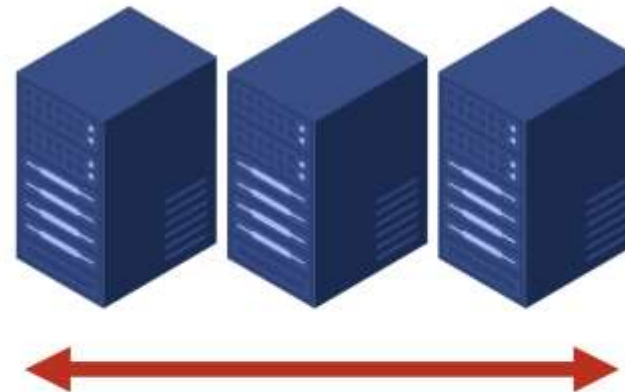
Vertical Scaling

Increase or decrease the capacity of existing services/instances.



Horizontal Scaling

Add more resources like virtual machines to your system to spread out the workload across them.



Lab : AWS Scaling

Amazon Route 53

Amazon Route 53

- Amazon Route 53 is a highly scalable and reliable Domain Name System (DNS) web service provided by Amazon Web Services (AWS).
- Its primary function is to help route end-user requests to internet applications by translating human-readable domain names into IP addresses.

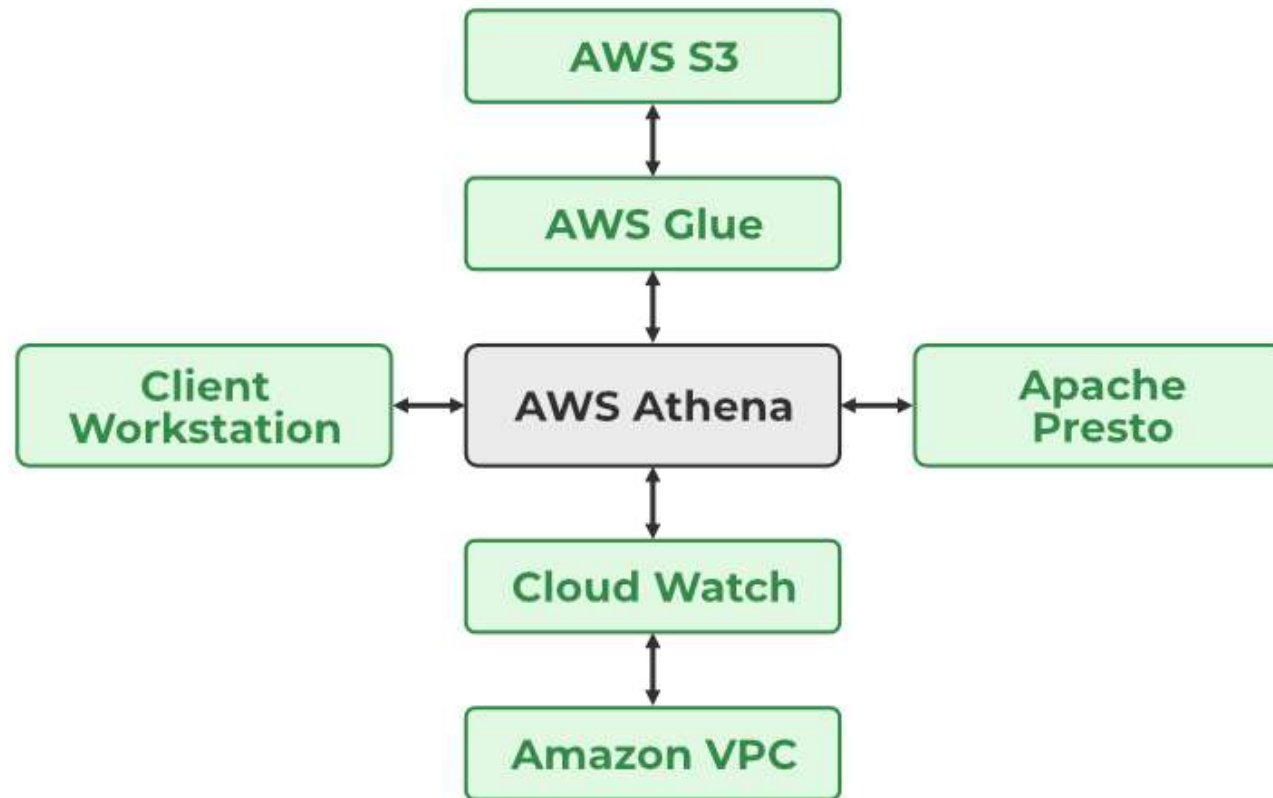
Services and features of Route53

- Domain registration
- DNS routing
- Health checking

What is AWS Athena

- AWS Athena is a serverless interactive query service that enables normal SQL data analysis in Amazon S3.
- Athena is based on Presto, a distributed SQL query engine, and it can query data in Amazon S3 fast using conventional SQL syntax.
- There is no infrastructure to handle with Athena, so you can focus on analyzing data at scale.

How AWS Athena Work



Features of AWS Athena

- **Serverless architecture** – **Athena** is a fully-managed service that does not require any infrastructure setup, management, or scaling.
- **Standard SQL support** – Since Athena supports ANSI SQL, users can query data in S3 using their existing SQL knowledge and tools.
- **Connection with the AWS ecosystem** – Athena interfaces with other AWS services such as Amazon S3, AWS Glue, and AWS Lambda, enabling customers to import and convert data from a variety of sources.
- **Cost-effective pricing model** – Athena's pricing approach is cost-effective since it costs customers based on the amount of data scanned by their queries, making it ideal for ad-hoc and exploratory queries.
- **Integration with BI tools** – Athena provides connectivity with major business intelligence tools like as Tableau, Power BI, and Amazon QuickSight, allowing users to build visualizations and reports.

AWS Athena Lab

Thank You